

Apache Kafka CLI Lab Guide

Step-by-step labs with full solutions - from installing Kafka on Windows to reading index files byte by byte. Every exercise runs on a three-broker Kafka 4.3.1 cluster in Docker.

TOPICS

PARTITIONS

SEGMENTS

PRODUCERS

CONSUMERS

GROUPS

LOG FILES

INDEXES

0 Setting Up Kafka on Windows

1 Start the Kafka Cluster

2 Topic Management from the Command Line

3 Partitions and Replication

4 Segments, Retention and Compaction

5 Producers: Publishing Events

6 Consumers, Consumer Properties and Offsets

7 Consumer Groups

8 Inside the Log Files

9 Index Files

10 Cheat Sheet and Troubleshooting

0

1

2

3

4

5

6

7

8

9

10

11

Lab 0 · Setting Up Kafka on Windows

What you will build

By the end of this lab book you will have run a real **three-broker Apache Kafka 4.3.1 cluster** in Docker on a Windows laptop, and used the command-line tools to manage topics, move data in and out, work with consumer groups, and look inside the files Kafka writes to disk.

Lab	Topic	What you do
0	Setup	Choose how to run Kafka on Windows, install Docker Desktop, get the lab files
1	Start the cluster	Start three brokers, check the KRaft quorum, look inside a broker
2	Topic management	Create, list, describe, configure, grow and delete topics
3	Partitions and replication	Where partitions live, broker failure, ISR, <code>min.insync.replicas</code> , leader election, reassignment
4	Segments, retention, compaction	Watch segments roll, get deleted by retention, and get compacted
5	Producers	Console producer, keys, headers, config files, acks, compression, performance tests
6	Consumers and offsets	Console consumer, formatting, reading from any offset, consumer properties, offset types
7	Consumer groups	Partition assignment, rebalancing, lag, resetting offsets, the new consumer protocol
8	Inside the log files	<code>kafka-dump-log</code> : batches, records, compression, internal topics, the metadata log
9	Index files	<code>.index</code> and <code>.timeindex</code> : format, lookups, density, verification
10	Cheat sheet	Every command from the labs on one page, plus troubleshooting

Every output in this book is real. It was captured from the lab cluster exactly as described here. Your IDs, timestamps and a few numbers (throughput, leader placement) will differ; the shape of the output will not. Very long outputs are shortened: `...` marks lines or fields left out, and columns are spaced for readability.

HOW THE LABS ARE WRITTEN

Each exercise has a **Task** (what to do, in plain words) followed by a **Solution** (the exact commands, the output you should see, and an explanation of what it means). Try the task first; open the solution when you are stuck or want to check.

0.1 Three ways to run Kafka on Windows

Option	How	Use it for
A. Docker Desktop (this lab book)	The official <code>apache/kafka</code> image, run by Docker Compose	Training, development, anything you want to reset in seconds
B. Native Windows	Unzip the Apache Kafka download, run the <code>bin\windows*.bat</code> scripts	Quick experiments only - see the warnings below
C. WSL2 (Linux on Windows)	Unzip the download inside Ubuntu on WSL2, run the <code>bin/*.sh</code> scripts	Developers who prefer a Linux shell

All exercises in this book use option A. Option B is described in section 0.3 for reference, because trainees often ask about it.

0.2 Install Docker Desktop

Requirements

- Windows 10 (22H2) or Windows 11, 64-bit
- Hardware virtualisation enabled in the BIOS (most laptops have it on)
- At least 8 GB of RAM; the lab cluster itself uses about 2 GB
- About 3 GB of free disk space for the image and data

Steps

Step 1 - Turn on WSL2. Open PowerShell *as Administrator* and run the command below. Restart when asked. (If WSL is already installed, it does nothing harmful.)

```
wsl --install
```

POWERSHELL

Step 2 - Install Docker Desktop. Download *Docker Desktop for Windows* from <https://www.docker.com/products/docker-desktop/> and run the installer. Keep the option **"Use WSL 2 instead of Hyper-V"** ticked.

Step 3 - Start Docker Desktop from the Start menu and wait until the status bar says *Engine running*.

Step 4 - Check it works. Open a normal (non-admin) PowerShell window:

```
docker version --format 'Client {{.Client.Version}} / Server {{.Server.Version}}
({{.Server.Os}}/{{.Server.Arch}})'
docker compose version
wsl --status
```

POWERSHELL

```
Client 29.8.0 / Server 29.8.0 (linux/amd64)
Docker Compose version v5.5.1
Default Distribution: Ubuntu
Default Version: 2
```

OUTPUT

`Server ... (linux/amd64)` means Docker is running Linux containers through WSL2 - exactly what the Kafka image needs.

Step 5 - Download the Kafka image (about 240 MB compressed, 690 MB on disk):

```
docker pull apache/kafka:4.3.1
docker images apache/kafka
```

POWERSHELL

IMAGE	ID	DISK USAGE	CONTENT SIZE
apache/kafka:4.3.1	77e3df905404	686MB	239MB

OUTPUT

WHY PIN THE VERSION?

`apache/kafka:latest` changes whenever Apache releases a new version. Pinning `4.3.1` (the latest release when this book was written) means every trainee runs exactly the same broker, and every output in this book matches.

0.3 Reference: running Kafka natively on Windows

NOT USED IN THE EXERCISES

This section is for reference only. It is not part of the labs and was not run for this book. Apache Kafka does not support Windows as a production platform, and native Windows brokers have a long history of **file-locking problems**: deleting a topic, or retention deleting old segments, can fail with `AccessDeniedException` and stop the broker. Use Docker (or WSL2) for anything beyond a quick try.

If you still want to see it work natively:

Step 1 - Install Java 17 or newer. Kafka 4.x brokers need Java 17+ (clients need Java 11+). Check with `java -version`.

Step 2 - Download the binary release `kafka_2.13-4.3.1.tgz` from `https://kafka.apache.org/downloads`.

Step 3 - Extract it to a short path such as `C:\kafka`. Long paths break the `.bat` scripts, which build a very long classpath.

POWERSHELL

```
mkdir C:\kafka
tar -xzf kafka_2.13-4.3.1.tgz -C C:\kafka --strip-components 1
```

Step 4 - Point the data directory somewhere sensible. In `C:\kafka\config\server.properties` change `log.dirs` to `log.dirs=C:/kafka/data` (forward slashes avoid escaping problems).

Step 5 - Format the storage once, then start the broker (KRaft, single node, no ZooKeeper):

POWERSHELL

```
cd C:\kafka
$id = bin\windows\kafka-storage.bat random-uuid
bin\windows\kafka-storage.bat format --standalone -t $id -c config\server.properties
bin\windows\kafka-server-start.bat config\server.properties
```

Step 6 - Use it from a second PowerShell window. Every tool in this book exists as a `.bat` file in `bin\windows\` with the same options:

POWERSHELL

```
bin\windows\kafka-topics.bat --bootstrap-server localhost:9092 --create --topic test
```

0.4 The lab files

The lab folder is `Apache_Kafka_Training\labs\kafka_cli_lab`:

OUTPUT

```
kafka_cli_lab\
  docker-compose.yml      the 3-broker cluster (and optional Kafka UI)
  lab\
    config\
      producer.properties  used in Lab 5
      consumer.properties  used in Lab 6
      increase-rf.json      used in Lab 3 (reassignment)
    data\
      orders.txt            10 keyed sample orders (Labs 3, 5)
      payloads.txt          500 JSON orders for the compression test (Lab 5)
      host_client_check.py  optional: talk to the cluster from Windows (Lab 1)
      Kafka_CLI_Lab_Guide.pdf this book
```

The `lab` folder is mounted into every broker container as `/lab`, so files you put there on Windows are visible inside the containers.

The cluster, explained

`docker-compose.yml` starts three identical containers, `kafka1`, `kafka2` and `kafka3`. The settings that matter:

Setting	Value	Meaning
<code>image</code>	<code>apache/kafka:4.3.1</code>	Official Apache image, Kafka 4.3.1
<code>KAFKA_PROCESS_ROLES</code>	<code>broker,controller</code>	Each node is a broker and a KRaft controller ("combined mode"). Fine for a laptop; production runs separate controllers.
<code>KAFKA_CONTROLLER_QUORUM_VOTERS</code>	<code>1@kafka1:9093,2@kafka2:9093,3@kafka3:9093</code>	The three nodes vote on cluster metadata (Raft). A majority - 2 of 3 - must be up.
<code>CLUSTER_ID</code>	<code>mXRdmJ0FRR-s1q-6q0Gqew</code>	Every node must be formatted with the same cluster ID
<code>KAFKA_LISTENERS</code>	<code>PLAINTEXT://:9092, CONTROLLER://:9093, EXTERNAL://:19092</code>	Three sockets: brokers and containers use 9092, controllers use 9093, Windows uses 19092 / 29092 / 39092
<code>KAFKA_ADVERTISED_LISTENERS</code>	<code>PLAINTEXT://kafka1:9092, EXTERNAL://localhost:19092</code>	The addresses the broker tells clients to use, per listener
<code>KAFKA_DEFAULT_REPLICATION_FACTOR</code>	<code>3</code>	New topics get 3 copies unless told otherwise
<code>KAFKA_MIN_INSYNC_REPLICAS</code>	<code>2</code>	With <code>acks=all</code> , a write needs 2 in-sync copies
<code>KAFKA_NUM_PARTITIONS</code>	<code>3</code>	New topics get 3 partitions unless told otherwise
<code>KAFKA_AUTO_CREATE_TOPICS_ENABLE</code>	<code>false</code>	Producing to an unknown topic fails instead of silently creating it
<code>KAFKA_LOG_RETENTION_CHECK_INTERVAL_MS</code>	<code>30000</code>	Lab only: check retention every 30 s instead of every 5 min
<code>KAFKA_HEAP_OPTS</code>	<code>-Xms512m -Xmx512m</code>	Keep each JVM small on a laptop
volumes	<code>kafka1_data:/var/lib/kafka/data</code>	Each broker's partitions survive a container restart

WHY ADVERTISED LISTENERS MATTER

A client connects to *any* broker in `bootstrap.servers`, gets back the cluster metadata, and then connects to the **advertised** address of each broker it needs. Inside Docker the brokers advertise `kafka1:9092`; to Windows they

advertise `localhost:19092`. Get this wrong and clients connect once, then fail with "connection refused" to an address they cannot reach.

0.5 Conventions used in this book

Code blocks are labelled:

POWERSHELL

```
# PowerShell on Windows - run in the lab folder
docker compose ps
```

KAFKA1 SHELL

```
# inside a broker container - run after: docker exec -it kafka1 bash
kafka-topics.sh --bootstrap-server kafka1:9092 --list
```

OUTPUT

what you should see

Opening a shell in a broker. Most commands in this book run *inside* the `kafka1` container, where every Kafka tool is on the `PATH`:

POWERSHELL

```
docker exec -it kafka1 bash
```

Type `exit` to leave. You can also run a single command without opening a shell:

POWERSHELL

```
docker exec kafka1 kafka-topics.sh --bootstrap-server kafka1:9092 --list
```

GIT BASH USERS

Git Bash rewrites paths that start with `/` (it turns `/lab/data` into `C:/Program Files/Git/lab/data`). Use PowerShell, or run `export MSYS_NO_PATHCONV=1` once in your Git Bash window.

Lab 1 · Start the Kafka Cluster

Goal: start the three brokers, prove they formed one cluster, and see what a broker keeps on disk.

Exercise 1.1 · Start the cluster

Task: start all three brokers in the background and wait until every container reports *healthy*.

✓ SOLUTION

```
cd D:\trainings\Apache_Kafka_Training\labs\kafka_cli_lab
docker compose up -d
docker compose ps
```

POWERSHELL

NAME	IMAGE	STATUS	PORTS
kafka1	apache/kafka:4.3.1	Up 22 seconds (healthy)	0.0.0.0:19092->19092/tcp
kafka2	apache/kafka:4.3.1	Up 22 seconds (healthy)	0.0.0.0:29092->29092/tcp
kafka3	apache/kafka:4.3.1	Up 22 seconds (healthy)	0.0.0.0:39092->39092/tcp

OUTPUT

The first start takes 15-30 seconds. `(health: starting)` becomes `(healthy)` once the health check - `kafka-broker-api-versions.sh` against the broker - succeeds. Each broker publishes one port to Windows.

To see a broker's own log, and the line that says it is ready:

```
docker compose logs kafka1 | Select-String "Kafka Server started"
```

POWERSHELL

```
[2026-09-27 11:50:34,672] INFO [KafkaRaftServer nodeId=1] Kafka Server started (kafka.server.KafkaRaftServer)
```

OUTPUT

`KafkaRaftServer` confirms KRaft mode - there is no ZooKeeper anywhere in this cluster.

Exercise 1.2 · Prove it is one cluster

Task: from inside `kafka1`, find the Kafka version, the cluster ID, which node leads the KRaft quorum, and the list of brokers.

✓ SOLUTION

Open a shell in the first broker:

```
docker exec -it kafka1 bash
```

POWERSHELL

Version and cluster ID:

```
kafka-topics.sh --version
kafka-cluster.sh cluster-id --bootstrap-server kafka1:9092
```

KAFKA1 SHELL

```
4.3.1
Cluster ID: mXRdmJ0FRR-s1q-6q0Gqew
```

OUTPUT

The KRaft quorum - the three controllers that keep the cluster's metadata:

```
kafka-metadata-quorum.sh --bootstrap-server kafka1:9092 describe --status
```

KAFKA1 SHELL

OUTPUT

```
ClusterId:      mXRdmJ0FRR-s1q-6q0Gqew
LeaderId:       2
LeaderEpoch:   1
HighWatermark:  88
MaxFollowerLag: 0
MaxFollowerLagTimeMs: 434
CurrentVoters:  [{"id": 1, "endpoints": ["CONTROLLER://kafka1:9093"]}, {"id": 2, "endpoints": ["CONTROLLER://kafka2:9093"]}, {"id": 3, "endpoints": ["CONTROLLER://kafka3:9093"]}
CurrentObservers: []
```

Field	Meaning
LeaderId	The active controller - here node 2. Any node can win the election.
LeaderEpoch	Goes up by one every time a new controller leader is elected
HighWatermark	How many metadata records all voters have agreed on
CurrentVoters	The Raft voters - all three nodes

```
kafka-metadata-quorum.sh --bootstrap-server kafka1:9092 describe --replication
```

KAFKA1 SHELL

OUTPUT

NodeId	DirectoryId	LogEndOffset	Lag	LastFetchTimestamp	LastCaughtUpTimestamp	Status
2	AAAAAAAAAAAAAAAAAAAAA	94	0	1790509870079	1790509870079	Leader
1	AAAAAAAAAAAAAAAAAAAAA	94	0	1790509869709	1790509869709	Follower
3	AAAAAAAAAAAAAAAAAAAAA	94	0	1790509869709	1790509869709	Follower

All three have the same **LogEndOffset** and **Lag 0**: the followers have copied every metadata record.

The brokers - every broker answers this with its supported API versions; we only keep the first line of each:

```
kafka-broker-api-versions.sh --bootstrap-server kafka1:9092 | grep -E '^kafka'
```

KAFKA1 SHELL

OUTPUT

```
kafka3:9092 (id: 3 rack: null isFenced: false) -> (
kafka1:9092 (id: 1 rack: null isFenced: false) -> (
kafka2:9092 (id: 2 rack: null isFenced: false) -> (
```

isFenced: false means the controller has accepted the broker and it can lead partitions.

Exercise 1.3 · Look inside a broker

Task: list the broker's data directory and read the file that identifies it.

✓ SOLUTION

```
ls -l /var/lib/kafka/data
cat /var/lib/kafka/data/meta.properties
```

KAFKA1 SHELL

OUTPUT

```
drwxr-xr-x 2 appuser appuser 4096 Sep 27 11:50 __cluster_metadata-0
-rw-r--r-- 1 appuser appuser 411 Sep 27 11:50 bootstrap.checkpoint
-rw-r--r-- 1 appuser appuser 0 Sep 27 11:50 cleaner-offset-checkpoint
-rw-r--r-- 1 appuser appuser 4 Sep 27 11:51 log-start-offset-checkpoint
-rw-r--r-- 1 appuser appuser 122 Sep 27 11:50 meta.properties
-rw-r--r-- 1 appuser appuser 4 Sep 27 11:51 recovery-point-offset-checkpoint
-rw-r--r-- 1 appuser appuser 0 Sep 27 11:50 replication-offset-checkpoint
#
#Sun Sep 27 11:50:28 GMT 2026
cluster.id=mXRdmJ0FRR-s1q-6q0Gqew
directory.id=W6ZXRWs45VFCVynSoUEXg
node.id=1
version=1
```

Entry	What it is
<code>meta.properties</code>	Written by <code>kafka-storage.sh format</code> . A broker refuses to start if its <code>cluster.id</code> does not match the cluster's.
<code>__cluster_metadata-0/</code>	The KRaft metadata log : every topic, partition, config and broker registration, as records. Lab 8 decodes it.
<code>bootstrap.checkpoint</code>	The metadata the cluster was formatted with (feature versions)
<code>*-checkpoint</code> files	Per-partition bookkeeping: where each log starts, how far it is safely flushed, how far each replica has replicated, how far compaction got

Once you create topics, each partition gets its own directory here, named `<topic>-<partition>`.

Exercise 1.4 · (Optional) Connect from Windows

Task: prove the cluster is reachable from programs running on Windows, not only from inside Docker.

✓ SOLUTION

`lab\host_client_check.py` lists the brokers and topics, writes one message and reads three back, using `localhost:19092,localhost:29092,localhost:39092`. It needs Python and the `confluent-kafka` package. Run it after Lab 3, when the `orders` topic has data:

POWERSHELL

```
pip install confluent-kafka
python lab\host_client_check.py
```

OUTPUT

```
brokers : ['1=localhost:19092', '2=localhost:29092', '3=localhost:39092']
topics  : ['audit-log', 'customer-profile', 'orders', 'payments', ...]
written : orders-0 @ offset 6
read    : 2 0 carol {"order":2003,"item":"mouse","amount":18
read    : 2 1 erin {"order":2005,"item":"webcam","amount":7
read    : 2 2 carol {"order":2009,"item":"desk lamp","amount
```

The brokers are listed with their **advertised** EXTERNAL addresses - `localhost:19092` and so on - which is why a Windows program can follow them.

Exercise 1.5 · Stop, start and reset

Task: learn the three ways to stop the lab.

✓ SOLUTION

Command (PowerShell, lab folder)	Effect
<code>docker compose stop</code>	Stops the containers; <code>docker compose start</code> brings them back as they were
<code>docker compose down</code>	Removes the containers but keeps the data volumes; <code>up -d</code> restores all topics and messages
<code>docker compose down -v</code>	Removes containers and volumes - every topic and message is gone. Use it to start the labs again from scratch.
<code>docker compose --profile ui up -d</code>	Also starts Kafka UI at <code>http://localhost:8080</code> - a web view of the same cluster, handy for checking your CLI work

Lab 2 · Topic Management from the Command Line

Goal: use `kafka-topics.sh` and `kafka-configs.sh` to create, inspect, configure, grow and delete topics - and learn the errors you will meet on the way.

All commands in this lab run inside `kafka1` (`docker exec -it kafka1 bash`).

THE TWO TOPIC TOOLS

- `kafka-topics.sh` - create, list, describe, add partitions, delete.
- `kafka-configs.sh` - read and change settings of topics, brokers, users and clients, without restarting anything.

Both talk to the cluster through `--bootstrap-server`. Older books show `--zookeeper`; that option no longer exists in Kafka 4.x.

Exercise 2.1 · Create topics

Task: create three topics:

- `orders` - 3 partitions, replication factor 3
- `payments` - 6 partitions, replication factor 3, keep data for 1 day, allow messages up to 2 MB
- `audit-log` - using only the cluster defaults

✓ SOLUTION

```
kafka-topics.sh --bootstrap-server kafka1:9092 --create --topic orders \
  --partitions 3 --replication-factor 3
```

KAFKA1 SHELL

```
Created topic orders.
```

OUTPUT

```
kafka-topics.sh --bootstrap-server kafka1:9092 --create --topic payments \
  --partitions 6 --replication-factor 3 \
  --config retention.ms=86400000 --config max.message.bytes=2097152
```

KAFKA1 SHELL

```
Created topic payments.
```

OUTPUT

```
kafka-topics.sh --bootstrap-server kafka1:9092 --create --topic audit-log
```

KAFKA1 SHELL

```
Created topic audit-log.
```

OUTPUT

- `--config key=value` sets a **topic-level override** at creation time; repeat it for each setting. `86400000` ms is 24 hours; `2097152` bytes is 2 MB.
- `audit-log` got the broker defaults from `docker-compose.yml`: `num.partitions=3` and `default.replication.factor=3`.

Exercise 2.2 · The errors you will meet

Task: try each of these and read the error:

1. Create `orders` again.
2. Create `orders` again, but so that the command succeeds if it already exists.
3. Create a topic with replication factor 4.
4. Create `user.events`, then `user_events`.

✓ SOLUTION

1 - Topic already exists:

```
kafka-topics.sh --bootstrap-server kafka1:9092 --create --topic orders --partitions 3
```

KAFKA1 SHELL

```
Error while executing topic command : Topic 'orders' already exists.
[2026-09-27 11:51:27,587] ERROR org.apache.kafka.common.errors.TopicExistsException: Topic 'orders' already exists.
```

OUTPUT

2 - `--if-not-exists` makes creation idempotent - ideal in scripts that may run twice:

```
kafka-topics.sh --bootstrap-server kafka1:9092 --create --topic orders --partitions 3 --if-not-exists
echo "exit code: $?"
```

KAFKA1 SHELL

```
exit code: 0
```

OUTPUT

3 - More copies than brokers:

```
kafka-topics.sh --bootstrap-server kafka1:9092 --create --topic too-many-copies --replication-factor 4
```

KAFKA1 SHELL

```
Error while executing topic command : Unable to replicate the partition 4 time(s): The target
replication factor of 4 cannot be reached because only 3 broker(s) are registered or some brokers
have all their log directories cordoned.
```

OUTPUT

Each copy of a partition must be on a different broker, so the replication factor can never exceed the number of brokers.

4 - Names that collide:

```
kafka-topics.sh --bootstrap-server kafka1:9092 --create --topic user.events
kafka-topics.sh --bootstrap-server kafka1:9092 --create --topic user_events
```

KAFKA1 SHELL

```
WARNING: Due to limitations in metric names, topics with a period ('.') or underscore ('_') could collide. To
avoid issues it is best to use either, but not both.
Created topic user.events.
WARNING: Due to limitations in metric names, topics with a period ('.') or underscore ('_') could collide. To
avoid issues it is best to use either, but not both.
Error while executing topic command : Topic 'user_events' collides with existing topic: user.events
```

OUTPUT

Kafka's metrics replace `.` with `_`, so these two names would share one set of metrics. Pick one separator for all your topic names (this book uses `-`).

Topic naming rules: letters, digits, `.`, `_` and `-`; at most 249 characters; not `.` or `..`.

Exercise 2.3 · Choose where replicas go

Task: create a topic `pinned` with 3 partitions and 2 replicas each, placed like this: partition 0 on brokers 1 and 2, partition 1 on brokers 2 and 3, partition 2 on brokers 3 and 1.

✓ SOLUTION

`--replica-assignment` takes a comma-separated list, one entry per partition; each entry is a colon-separated list of broker IDs. The **first** broker in each entry is the preferred leader. Do not pass `--partitions` or `--replication-factor` with it - both follow from the list.

```
kafka-topics.sh --bootstrap-server kafka1:9092 --create --topic pinned \
  --replica-assignment 1:2,2:3,3:1
kafka-topics.sh --bootstrap-server kafka1:9092 --describe --topic pinned
```

OUTPUT

```
Created topic pinned.
Topic: pinned TopicId: MTyokh2qRhWcC_bIsZVkgk PartitionCount: 3 ReplicationFactor: 2 Configs:
min.insync.replicas=2
  Topic: pinned Partition: 0 Leader: 1 Replicas: 1,2 Isr: 1,2 Elr: LastKnownElr:
  Topic: pinned Partition: 1 Leader: 2 Replicas: 2,3 Isr: 2,3 Elr: LastKnownElr:
  Topic: pinned Partition: 2 Leader: 3 Replicas: 3,1 Isr: 3,1 Elr: LastKnownElr:
```

Pinning is how you control placement exactly - for example, to put a single-partition topic on a known broker so you can look at its files (Labs 4, 8 and 9 do this).

Exercise 2.4 · List and describe

Task: list all topics, then describe `orders` and `payments`. Explain every column of the describe output.

✓ SOLUTION

```
kafka-topics.sh --bootstrap-server kafka1:9092 --list
```

OUTPUT

```
audit-log
orders
payments
pinned
user.events
```

```
kafka-topics.sh --bootstrap-server kafka1:9092 --describe --topic orders
```

OUTPUT

```
Topic: orders TopicId: ywkgJzfISRGp0UN2q1Y88A PartitionCount: 3 ReplicationFactor: 3 Configs:
min.insync.replicas=2
  Topic: orders Partition: 0 Leader: 3 Replicas: 3,1,2 Isr: 3,1,2 Elr: LastKnownElr:
  Topic: orders Partition: 1 Leader: 1 Replicas: 1,2,3 Isr: 1,2,3 Elr: LastKnownElr:
  Topic: orders Partition: 2 Leader: 2 Replicas: 2,3,1 Isr: 2,3,1 Elr: LastKnownElr:
```

KAFKA1 SHELL

```
kafka-topics.sh --bootstrap-server kafka1:9092 --describe --topic payments
```

OUTPUT

```
Topic: payments TopicId: r4Sp6_JBST-S1r30VVuQNQ PartitionCount: 6 ReplicationFactor: 3 Configs:
min.insync.replicas=2,retention.ms=86400000,max.message.bytes=2097152
Topic: payments Partition: 0 Leader: 2 Replicas: 2,3,1 Isr: 2,3,1 Elr: LastKnownElr:
Topic: payments Partition: 1 Leader: 3 Replicas: 3,1,2 Isr: 3,1,2 Elr: LastKnownElr:
Topic: payments Partition: 2 Leader: 1 Replicas: 1,2,3 Isr: 1,2,3 Elr: LastKnownElr:
Topic: payments Partition: 3 Leader: 3 Replicas: 3,1,2 Isr: 3,1,2 Elr: LastKnownElr:
Topic: payments Partition: 4 Leader: 1 Replicas: 1,2,3 Isr: 1,2,3 Elr: LastKnownElr:
Topic: payments Partition: 5 Leader: 2 Replicas: 2,3,1 Isr: 2,3,1 Elr: LastKnownElr:
```

Column	Meaning
<code>TopicId</code>	A permanent unique ID. If you delete a topic and create one with the same name, it gets a new ID.
<code>PartitionCount</code> / <code>ReplicationFactor</code>	Number of partitions / copies of each partition
<code>Configs</code>	Settings that differ from Kafka's built-in defaults (see the note below)
<code>Leader</code>	The broker that serves all reads and writes for this partition
<code>Replicas</code>	Every broker holding a copy. The first one is the <i>preferred leader</i> .
<code>Isr</code>	In-sync replicas - the copies that are fully caught up with the leader
<code>Elr</code>	Eligible leader replicas (new in Kafka 4.x) - replicas that left the ISR but are guaranteed to hold all committed data, so they may still become leader
<code>LastKnownElr</code>	ELR members that were also fenced - the last resort if everything else fails

Notice how Kafka spreads the work: each broker is the leader of an equal share of partitions, and each broker holds a copy of every partition (replication factor 3 on 3 brokers).

WHY `MIN.INSYNC.REPLICAS=2` APPEARS ON EVERY TOPIC

We never set it per topic. Kafka 4.x records the cluster-wide `min.insync.replicas` as a *dynamic cluster default* (the ELR feature needs it), and `--describe` shows it on every topic. `kafka-configs.sh --describe` in Exercise 2.6 shows where it comes from.

Exercise 2.5 · Filter the describe output

Task: show only the topics that have their own settings.

✓ SOLUTION

KAFKA1 SHELL

```
kafka-topics.sh --bootstrap-server kafka1:9092 --describe --topics-with-overrides
```

OUTPUT

```
Topic: orders TopicId: ywkgJzfISRGp0UN2q1Y88A PartitionCount: 3 ReplicationFactor: 3 Configs:
min.insync.replicas=2
Topic: pinned TopicId: MTyokh2qRhWcC_bIsZVvkg PartitionCount: 3 ReplicationFactor: 2 Configs:
min.insync.replicas=2
Topic: user.events TopicId: 1u3C0AHNTPG-di0gCIC6BQ PartitionCount: 3 ReplicationFactor: 3 Configs:
min.insync.replicas=2
```

```
Topic: audit-log   TopicId: RUzgBoJcRhmMC_psWmYb3w  PartitionCount: 3  ReplicationFactor: 3  Configs:
min.insync.replicas=2
Topic: payments   TopicId: r4Sp6_JBST-S1r30VVuQNQ  PartitionCount: 6  ReplicationFactor: 3  Configs:
min.insync.replicas=2,retention.ms=86400000,max.message.bytes=2097152
```

The other describe filters - you will use them in Lab 3 when a broker goes down:

Option	Shows partitions that ...
<code>--under-replicated-partitions</code>	have fewer in-sync replicas than replicas
<code>--under-min-isr-partitions</code>	have fewer in-sync replicas than <code>min.insync.replicas</code> - <code>acks=all</code> writes fail
<code>--at-min-isr-partitions</code>	have exactly <code>min.insync.replicas</code> in sync - one more failure and writes stop
<code>--unavailable-partitions</code>	have no leader at all - no reads or writes
<code>--exclude-internal</code>	(with any of the above) hide <code>__consumer_offsets</code> and other internal topics

Exercise 2.6 · Read and change topic settings

Task:

1. Show the settings `payments` overrides, then *all* settings of `orders` that deal with retention, segments and compression.
2. Give `audit-log` a 7-day retention and `zstd` compression, then remove the compression setting again.
3. Try to set `segment.bytes=1000` on `audit-log`.
4. Show the cluster-wide broker defaults.

✓ SOLUTION

1 - Overrides only, then everything:

```
kafka-configs.sh --bootstrap-server kafka1:9092 --describe --entity-type topics --entity-name payments
```

OUTPUT

```
Dynamic configs for topic payments are:
max.message.bytes=2097152 sensitive=false synonyms={DYNAMIC_TOPIC_CONFIG:max.message.bytes=2097152,
DEFAULT_CONFIG:message.max.bytes=1048588}
retention.ms=86400000 sensitive=false synonyms={DYNAMIC_TOPIC_CONFIG:retention.ms=86400000}
```

`synonyms` shows every level the value could come from, highest priority first: the topic's own setting (`DYNAMIC_TOPIC_CONFIG`), then the broker's (`message.max.bytes`), then the built-in default.

```
kafka-configs.sh --bootstrap-server kafka1:9092 --describe --entity-type topics --entity-name orders --
all \
| grep -E 'cleanup.policy|retention|segment|index.interval|min.insync|max.message|compression.type'
```

OUTPUT

```
cleanup.policy=delete sensitive=false synonyms={DEFAULT_CONFIG:log.cleanup.policy=delete}
compression.type=producer sensitive=false synonyms={DEFAULT_CONFIG:compression.type=producer}
delete.retention.ms=86400000 sensitive=false synonyms=
{DEFAULT_CONFIG:log.cleaner.delete.retention.ms=86400000}
file.delete.delay.ms=60000 sensitive=false synonyms={DEFAULT_CONFIG:log.segment.delete.delay.ms=60000}
index.interval.bytes=4096 sensitive=false synonyms={DEFAULT_CONFIG:log.index.interval.bytes=4096}
local.retention.bytes=-2 sensitive=false synonyms={DEFAULT_CONFIG:log.local.retention.bytes=-2}
local.retention.ms=-2 sensitive=false synonyms={DEFAULT_CONFIG:log.local.retention.ms=-2}
max.message.bytes=1048588 sensitive=false synonyms={DEFAULT_CONFIG:message.max.bytes=1048588}
```

```
min.insync.replicas=2 sensitive=false synonyms={DYNAMIC_DEFAULT_BROKER_CONFIG:min.insync.replicas=2,
STATIC_BROKER_CONFIG:min.insync.replicas=2, DEFAULT_CONFIG:min.insync.replicas=1}
retention.bytes=-1 sensitive=false synonyms={DEFAULT_CONFIG:log.retention.bytes=-1}
retention.ms=604800000 sensitive=false synonyms={}
segment.bytes=1073741824 sensitive=false synonyms={DEFAULT_CONFIG:log.segment.bytes=1073741824}
segment.index.bytes=10485760 sensitive=false synonyms={DEFAULT_CONFIG:log.index.size.max.bytes=10485760}
segment.jitter.ms=0 sensitive=false synonyms={}
segment.ms=604800000 sensitive=false synonyms={}
```

The important defaults, which later labs change:

Setting	Default	Meaning
<code>cleanup.policy</code>	<code>delete</code>	Old data is deleted by age/size. <code>compact</code> keeps the latest value per key (Lab 4).
<code>retention.ms</code>	604800000 (7 days)	Delete segments older than this
<code>retention.bytes</code>	-1 (no limit)	Delete old segments when a partition grows beyond this
<code>segment.bytes</code>	1073741824 (1 GB)	Start a new segment file when the current one reaches this size
<code>segment.ms</code>	604800000 (7 days)	... or when it is this old
<code>index.interval.bytes</code>	4096	Add an index entry roughly every 4 KB of log (Lab 9)
<code>segment.index.bytes</code>	10485760 (10 MB)	Maximum size of each index file (Lab 9)
<code>compression.type</code>	<code>producer</code>	Keep whatever compression the producer used
<code>min.insync.replicas</code>	2 (our cluster)	Copies that must confirm an <code>acks=all</code> write

2 - Add settings, then remove one:

```
kafka-configs.sh --bootstrap-server kafka1:9092 --alter --entity-type topics --entity-name audit-log \
--add-config retention.ms=604800000,cleanup.policy=delete,compression.type=zstd
kafka-configs.sh --bootstrap-server kafka1:9092 --describe --entity-type topics --entity-name audit-log
```

OUTPUT

```
Completed updating config for topic audit-log.
Dynamic configs for topic audit-log are:
  cleanup.policy=delete sensitive=false synonyms={DYNAMIC_TOPIC_CONFIG:cleanup.policy=delete,
DEFAULT_CONFIG:log.cleanup.policy=delete}
  compression.type=zstd sensitive=false synonyms={DYNAMIC_TOPIC_CONFIG:compression.type=zstd,
DEFAULT_CONFIG:compression.type=producer}
  retention.ms=604800000 sensitive=false synonyms={DYNAMIC_TOPIC_CONFIG:retention.ms=604800000}
```

```
kafka-configs.sh --bootstrap-server kafka1:9092 --alter --entity-type topics --entity-name audit-log \
--delete-config compression.type
```

OUTPUT

```
Completed updating config for topic audit-log.
```

Changes take effect immediately on all brokers - no restart. `--delete-config` removes the override, so the topic falls back to the broker default.

3 - Values are validated:

```
kafka-configs.sh --bootstrap-server kafka1:9092 --alter --entity-type topics --entity-name audit-log \
```

```
--add-config segment.bytes=1000
```

OUTPUT

```
Caused by: org.apache.kafka.common.errors.InvalidConfigurationException: Invalid value 1000 for configuration
segment.bytes: Value must be at least 1048576
```

In Kafka 4.x a segment is at least 1 MB. Lab 4 uses exactly 1 MB to make segments roll quickly.

4 - Broker settings:

KAFKA1 SHELL

```
kafka-configs.sh --bootstrap-server kafka1:9092 --describe --entity-type brokers --entity-default
```

OUTPUT

```
Default configs for brokers in the cluster are:
min.insync.replicas=2 sensitive=false synonyms={DYNAMIC_DEFAULT_BROKER_CONFIG:min.insync.replicas=2}
```

KAFKA1 SHELL

```
kafka-configs.sh --bootstrap-server kafka1:9092 --describe --entity-type brokers --entity-name 1 --all \
| grep -E
'num.partitions|default.replication|log.retention.hours|log.segment.bytes|auto.create|min.insync'
```

OUTPUT

```
auto.create.topics.enable=false sensitive=false synonyms=
{STATIC_BROKER_CONFIG:auto.create.topics.enable=false, DEFAULT_CONFIG:auto.create.topics.enable=true}
default.replication.factor=3 sensitive=false synonyms={STATIC_BROKER_CONFIG:default.replication.factor=3,
DEFAULT_CONFIG:default.replication.factor=1}
log.retention.hours=168 sensitive=false synonyms={DEFAULT_CONFIG:log.retention.hours=168}
log.segment.bytes=1073741824 sensitive=false synonyms={DEFAULT_CONFIG:log.segment.bytes=1073741824}
min.insync.replicas=2 sensitive=false synonyms={DYNAMIC_DEFAULT_BROKER_CONFIG:min.insync.replicas=2,
STATIC_BROKER_CONFIG:min.insync.replicas=2, DEFAULT_CONFIG:min.insync.replicas=1}
num.partitions=3 sensitive=false synonyms={STATIC_BROKER_CONFIG:num.partitions=3,
DEFAULT_CONFIG:num.partitions=1}
```

`STATIC_BROKER_CONFIG` values came from `docker-compose.yml`; `DEFAULT_CONFIG` is Kafka's own default. Broker-level names start with `log.` where the topic-level name does not (`log.segment.bytes` vs `segment.bytes`).

Exercise 2.7 · Add partitions

Task: grow `audit-log` from 3 to 6 partitions. Then try to shrink it to 2.

✓ SOLUTION

KAFKA1 SHELL

```
kafka-topics.sh --bootstrap-server kafka1:9092 --alter --topic audit-log --partitions 6
kafka-topics.sh --bootstrap-server kafka1:9092 --describe --topic audit-log
```

OUTPUT

```
Topic: audit-log TopicId: RUzgBoJcRhMC_psWmYb3w PartitionCount: 6 ReplicationFactor: 3 Configs:
min.insync.replicas=2,cleanup.policy=delete,retention.ms=604800000
Topic: audit-log Partition: 0 Leader: 1 Replicas: 1,2,3 Isr: 1,2,3
Topic: audit-log Partition: 1 Leader: 2 Replicas: 2,3,1 Isr: 2,3,1
Topic: audit-log Partition: 2 Leader: 3 Replicas: 3,1,2 Isr: 3,1,2
Topic: audit-log Partition: 3 Leader: 3 Replicas: 3,1,2 Isr: 3,1,2
Topic: audit-log Partition: 4 Leader: 1 Replicas: 1,2,3 Isr: 1,2,3
Topic: audit-log Partition: 5 Leader: 2 Replicas: 2,3,1 Isr: 2,3,1
```

KAFKA1 SHELL

```
kafka-topics.sh --bootstrap-server kafka1:9092 --alter --topic audit-log --partitions 2
```

OUTPUT

```
Error while executing topic command : The topic audit-log currently has 6 partition(s); 2 would not be an increase.
```

ADDING PARTITIONS CHANGES WHERE KEYS GO

A keyed message goes to $\text{hash}(\text{key}) \bmod \text{number-of-partitions}$. Change the number of partitions and most keys map to a different partition from then on - new messages for customer `alice` may land in a different partition from her old ones, so per-key ordering breaks across the change. Kafka cannot remove partitions at all. Choose the partition count up front, with room to grow.

Exercise 2.8 · Delete topics

Task:

1. Delete `user.events`.
2. Create `tmp-a`, `tmp-b` and `tmp-c`, then delete all three with one command.
3. Delete a topic that does not exist - first so it fails, then so it does not.

✓ SOLUTION

1 - One topic:

KAFKA1 SHELL

```
kafka-topics.sh --bootstrap-server kafka1:9092 --delete --topic user.events
kafka-topics.sh --bootstrap-server kafka1:9092 --list
```

OUTPUT

```
audit-log
orders
payments
pinned
```

2 - Several topics by pattern. `--topic` accepts a regular expression for `--delete`, `--describe` and `--alter`. Quote it so the shell does not expand the `*`:

KAFKA1 SHELL

```
for t in tmp-a tmp-b tmp-c; do
  kafka-topics.sh --bootstrap-server kafka1:9092 --create --topic $t --partitions 1
done
kafka-topics.sh --bootstrap-server kafka1:9092 --delete --topic 'tmp-.*'
kafka-topics.sh --bootstrap-server kafka1:9092 --list
```

OUTPUT

```
Created topic tmp-a.
Created topic tmp-b.
Created topic tmp-c.
audit-log
orders
payments
pinned
```

3 - A topic that is not there:

KAFKA1 SHELL

```
kafka-topics.sh --bootstrap-server kafka1:9092 --delete --topic no-such-topic
```

OUTPUT

```
Error while executing topic command : Topic 'no-such-topic' does not exist as expected
```

KAFKA1 SHELL

```
kafka-topics.sh --bootstrap-server kafka1:9092 --delete --topic no-such-topic --if-exists
echo "exit code: $?"
```

OUTPUT

```
exit code: 0
```

DELETING A TOPIC IS PERMANENT

Deletion removes the topic from the metadata at once; the brokers then rename the partition directories with a `-delete` suffix and remove them in the background. There is no undo. In production, protect important topics with ACLs, and consider `delete.topic.enable=false` on the brokers.

Exercise 2.9 · Explore a topic's data

Task: without reading any messages, find out how much data each partition of `orders` holds, and how big it is on disk. (Do this after Exercise 3.1, which loads 10 orders.)

✓ SOLUTION

Offsets per partition. `kafka-get-offsets.sh` asks the leaders for an offset per partition. With no `--time` it returns the **latest** offset - the next offset that will be written:

KAFKA1 SHELL

```
kafka-get-offsets.sh --bootstrap-server kafka1:9092 --topic orders
```

OUTPUT

```
orders:0:4
orders:1:3
orders:2:3
```

Format: `topic:partition:offset`. Partition 0 holds offsets 0-3 (4 messages), and so on. `--time -2` gives the earliest offset, and `--time <epoch-ms>` the first offset at or after a timestamp - both are used in Labs 4, 6 and 9.

Size on disk, per broker. `kafka-log-dirs.sh` reports each partition directory's size and whether the copy lags behind:

KAFKA1 SHELL

```
kafka-log-dirs.sh --bootstrap-server kafka1:9092 --describe --topic-list orders --broker-list 1
```

OUTPUT

```
Querying brokers for log directories information
Received log directory information from brokers 1
{"brokers":[{"broker":1,"logDirs":[{"partitions":[{"partition":"orders-0","size":282,"offsetLag":0,"isFuture":false},{"partition":"orders-1","size":222,"offsetLag":0,"isFuture":false},{"partition":"orders-2","size":224,"offsetLag":0,"isFuture":false}],"error":null,"logDir":"/var/lib/kafka/data"}]}], "version":1}
```

`size` is in bytes. `offsetLag` is how far this copy is behind the leader. `isFuture: true` would mean the partition is being moved between disks on that broker.

Check yourself

1. Which command would you run to see *only* the partitions that are at risk because one more broker failure would stop writes?
2. A colleague ran `--alter --partitions 12` on a keyed topic that had 6. What changed for existing keys?
3. Where do the values in `synonyms={...}` come from, and which one wins?

✓ ANSWERS

1. `kafka-topics.sh --bootstrap-server kafka1:9092 --describe --at-min-isr-partitions`
2. Most keys now hash to a different partition. Old messages stay where they were; new messages for the same key may go elsewhere, so per-key order is only guaranteed within each "era".
3. From the topic override, the dynamic broker or cluster settings, the static broker file (`server.properties` / environment), and Kafka's built-in default - in that order. The first one listed wins.

Lab 3 · Partitions and Replication

Goal: see how keys map to partitions, where each copy of a partition lives, what happens when a broker dies, how `min.insync.replicas` protects your data, and how to move replicas around.

Exercise 3.1 · Keys decide the partition

Task: load the 10 sample orders from `/lab/data/orders.txt` into `orders`, using the customer name before the `:` as the key. Then read them back showing each message's partition. What do you notice?

✓ SOLUTION

The file on Windows is `lab\data\orders.txt`; inside the container it is `/lab/data/orders.txt`:

```
cat /lab/data/orders.txt
```

OUTPUT

```
alice:{"order":2001,"item":"keyboard","amount":45}
dave:{"order":2002,"item":"monitor","amount":210}
carol:{"order":2003,"item":"mouse","amount":18}
alice:{"order":2004,"item":"headset","amount":60}
erin:{"order":2005,"item":"webcam","amount":75}
frank:{"order":2006,"item":"dock","amount":140}
bob:{"order":2007,"item":"cable","amount":9}
dave:{"order":2008,"item":"chair","amount":320}
carol:{"order":2009,"item":"desk lamp","amount":35}
alice:{"order":2010,"item":"laptop stand","amount":55}
```

Feed the file to the console producer. `parse.key=true` splits each line at `key.separator` into key and value:

```
kafka-console-producer.sh --bootstrap-server kafka1:9092 --topic orders \
  --reader-property parse.key=true --reader-property key.separator=: \
  < /lab/data/orders.txt
```

```
kafka-console-consumer.sh --bootstrap-server kafka1:9092 --topic orders --from-beginning --max-messages
10 \
  --formatter-property print.partition=true --formatter-property print.key=true
```

OUTPUT

```
Partition:0 alice {"order":2001,"item":"keyboard","amount":45}
Partition:0 alice {"order":2004,"item":"headset","amount":60}
Partition:0 bob {"order":2007,"item":"cable","amount":9}
Partition:0 alice {"order":2010,"item":"laptop stand","amount":55}
Partition:1 dave {"order":2002,"item":"monitor","amount":210}
Partition:1 frank {"order":2006,"item":"dock","amount":140}
Partition:1 dave {"order":2008,"item":"chair","amount":320}
Partition:2 carol {"order":2003,"item":"mouse","amount":18}
Partition:2 erin {"order":2005,"item":"webcam","amount":75}
Partition:2 carol {"order":2009,"item":"desk lamp","amount":35}
Processed a total of 10 messages
```

- **Every message with the same key is in the same partition**, in the order it was written: all of alice's orders are in partition 0, in order 2001, 2004, 2010.
- Different keys can share a partition (alice and bob).
- The consumer reads partition by partition here, so the output is *not* in the original file order. Order is guaranteed **within a partition only**.

The Java producer chooses the partition as `murmur2(key) mod 3` - the same rule in every Java client, so the same key always lands in the same place.

Exercise 3.2 · Where do the copies live?

Task: list the partition directories for `orders` and `pinned` on each broker. Compare with `--describe`.

✓ SOLUTION

Run the same command in each container (from PowerShell):

```
docker exec kafka1 bash -c "hostname; ls -d /var/lib/kafka/data/orders-* /var/lib/kafka/data/pinned-*"
docker exec kafka2 bash -c "hostname; ls -d /var/lib/kafka/data/orders-* /var/lib/kafka/data/pinned-*"
docker exec kafka3 bash -c "hostname; ls -d /var/lib/kafka/data/orders-* /var/lib/kafka/data/pinned-*"
```

POWERSHELL

```
kafka1
/var/lib/kafka/data/orders-0
/var/lib/kafka/data/orders-1
/var/lib/kafka/data/orders-2
/var/lib/kafka/data/pinned-0
/var/lib/kafka/data/pinned-2
kafka2
/var/lib/kafka/data/orders-0
/var/lib/kafka/data/orders-1
/var/lib/kafka/data/orders-2
/var/lib/kafka/data/pinned-0
/var/lib/kafka/data/pinned-1
kafka3
/var/lib/kafka/data/orders-0
/var/lib/kafka/data/orders-1
/var/lib/kafka/data/orders-2
/var/lib/kafka/data/pinned-1
/var/lib/kafka/data/pinned-2
```

OUTPUT

- `orders` has replication factor 3 on 3 brokers, so **every broker has every partition**.
- `pinned` (replication factor 2) matches its `Replicas` column exactly: partition 0 on brokers 1 and 2, partition 1 on 2 and 3, partition 2 on 3 and 1.
- A replica is simply a directory named `<topic>-<partition>` on a broker's disk. Leaders and followers look identical on disk.

Exercise 3.3 · A broker fails

Task: create `strict-payments` (1 partition, replication factor 3, `min.insync.replicas=3`). Then stop broker 2 and answer:

- What happens to the leaders and the ISR of `orders`?
- Which partitions are under-replicated? Which are below their minimum ISR? Which are exactly at it?
- Can you still write to `orders` with `acks=all`? To `strict-payments`?

✓ SOLUTION

```
kafka-topics.sh --bootstrap-server kafka1:9092 --create --topic strict-payments \
  --partitions 1 --replication-factor 3 --config min.insync.replicas=3
```

KAFKA1 SHELL

Stop broker 2 **from PowerShell** (not inside a container):

```
docker stop kafka2
```

POWERSHELL

Wait about 10 seconds, then back in `kafka1`:

1 - Leaders and ISR:

```
kafka-topics.sh --bootstrap-server kafka1:9092 --describe --topic orders
```

KAFKA1 SHELL

OUTPUT

```
Topic: orders TopicId: ywkgJzfISRGp0UN2q1Y88A PartitionCount: 3 ReplicationFactor: 3 Configs:
min.insync.replicas=2
Topic: orders Partition: 0 Leader: 3 Replicas: 3,1,2 Isr: 3,1 Elr: LastKnownElr:
Topic: orders Partition: 1 Leader: 1 Replicas: 1,2,3 Isr: 1,3 Elr: LastKnownElr:
Topic: orders Partition: 2 Leader: 3 Replicas: 2,3,1 Isr: 3,1 Elr: LastKnownElr:
```

- Broker 2 was the leader of partition 2. The controller **elected broker 3** (the next in-sync replica) as the new leader, automatically, within seconds.
- Broker 2 dropped out of **every** ISR. `Replicas` does not change - broker 2 still *should* hold a copy; it just is not in sync.

2 - The health filters. Adding `--exclude-internal` hides the 50 partitions of `_consumer_offsets`, which are also affected:

```
kafka-topics.sh --bootstrap-server kafka1:9092 --describe --under-replicated-partitions --exclude-internal
```

KAFKA1 SHELL

OUTPUT

```
Topic: orders Partition: 0 Leader: 3 Replicas: 3,1,2 Isr: 3,1 Elr: LastKnownElr:
Topic: orders Partition: 1 Leader: 1 Replicas: 1,2,3 Isr: 1,3 Elr: LastKnownElr:
Topic: orders Partition: 2 Leader: 3 Replicas: 2,3,1 Isr: 3,1 Elr: LastKnownElr:
Topic: pinned Partition: 0 Leader: 1 Replicas: 1,2 Isr: 1 Elr: 2 LastKnownElr:
Topic: pinned Partition: 1 Leader: 3 Replicas: 2,3 Isr: 3 Elr: 2 LastKnownElr:
Topic: audit-log Partition: 0 Leader: 1 Replicas: 1,2,3 Isr: 1,3 Elr: LastKnownElr:
...
Topic: strict-payments Partition: 0 Leader: 1 Replicas: 1,2,3 Isr: 1,3 Elr: 2 LastKnownElr:
Topic: payments Partition: 0 Leader: 3 Replicas: 2,3,1 Isr: 3,1 Elr: LastKnownElr:
...
```

```
kafka-topics.sh --bootstrap-server kafka1:9092 --describe --under-min-isr-partitions
```

KAFKA1 SHELL

OUTPUT

```
Topic: pinned Partition: 0 Leader: 1 Replicas: 1,2 Isr: 1 Elr: 2 LastKnownElr:
Topic: pinned Partition: 1 Leader: 3 Replicas: 2,3 Isr: 3 Elr: 2 LastKnownElr:
Topic: strict-payments Partition: 0 Leader: 1 Replicas: 1,2,3 Isr: 1,3 Elr: 2 LastKnownElr:
```

```
kafka-topics.sh --bootstrap-server kafka1:9092 --describe --at-min-isr-partitions --topic orders
```

KAFKA1 SHELL

OUTPUT

```
Topic: orders Partition: 0 Leader: 3 Replicas: 3,1,2 Isr: 3,1 Elr: LastKnownElr:
Topic: orders Partition: 1 Leader: 1 Replicas: 1,2,3 Isr: 1,3 Elr: LastKnownElr:
Topic: orders Partition: 2 Leader: 3 Replicas: 2,3,1 Isr: 3,1 Elr: LastKnownElr:
```

- `orders` has 2 in-sync copies and needs 2: **at** the minimum - still writable, but one more failure stops `acks=all` writes.

- `pinned` (2 copies, needs 2) and `strict-payments` (3 copies, needs 3) are **below** the minimum.
- Broker 2 now appears under `Elr` for those partitions: when the ISR shrank below `min.insync.replicas`, Kafka kept note that broker 2 still holds all committed data and is eligible to lead again.

3 - Writing with `acks=all`:

```
echo 'grace:{ "order":2011,"item":"pen","amount":3}' | kafka-console-producer.sh \
  --bootstrap-server kafka1:9092 --topic orders \
  --reader-property parse.key=true --reader-property key.separator=: \
  --command-property acks=all
echo "exit code: $?"
```

KAFKA1 SHELL

```
exit code: 0
```

OUTPUT

```
echo 'payment-1' | kafka-console-producer.sh --bootstrap-server kafka1:9092 --topic strict-payments \
  --command-property acks=all --command-property delivery.timeout.ms=10000 --command-property
request.timeout.ms=5000
```

KAFKA1 SHELL

```
[2026-09-27 11:55:03,951] ERROR Error when sending message to topic strict-payments with key: null, value: 9
bytes with error: (org.apache.kafka.clients.producer.internals.ErrorLoggingCallback)
org.apache.kafka.common.errors.NotEnoughReplicasException: Messages are rejected since there are fewer in-sync
replicas than required.
```

OUTPUT

That is `min.insync.replicas` doing its job: the broker **refuses** an `acks=all` write it cannot store on enough copies, instead of accepting it and risking its loss.

WHAT ABOUT `ACKS=1`?

`min.insync.replicas` only applies to `acks=all`. With `acks=1` the write to `strict-payments` is accepted by the leader:

```
echo 'payment-1' | kafka-console-producer.sh --bootstrap-server kafka1:9092 --topic strict-payments --
command-property acks=1
kafka-get-offsets.sh --bootstrap-server kafka1:9092 --topic strict-payments
kafka-console-consumer.sh --bootstrap-server kafka1:9092 --topic strict-payments --from-beginning --
timeout-ms 5000
```

KAFKA1 SHELL

```
strict-payments:0:0
Processed a total of 0 messages
```

OUTPUT

The message is on the leader's disk, yet the partition still reports offset 0 and consumers see nothing. In Kafka 4.x the **high watermark** - the point up to which data counts as committed and is shown to consumers - only moves while the ISR is at least `min.insync.replicas`. The message becomes visible when broker 2 is back (next exercise). `acks=1` did not make the write safe; it only hid the problem from the producer.

The KRaft quorum noticed too. Node 2 is a controller voter, and it stopped fetching:

```
kafka-metadata-quorum.sh --bootstrap-server kafka1:9092 describe --replication
```

KAFKA1 SHELL

```
NodeId  DirectoryId  LogEndOffset  Lag  LastFetchTimestamp  LastCaughtUpTimestamp  Status
1        AAAAAAAAAAAAAAAAAAAAAA  755          0    1790510114975      1790510114975      Leader
```

OUTPUT

2	AAAAAAAAAAAAAAAAAAAA	-1	756	-1	-1	Follower
3	AAAAAAAAAAAAAAAAAAAA	755	0	1790510114711	1790510114711	Follower

DO NOT STOP TWO BROKERS IN THIS LAB

Each node here is also a controller, and Raft needs a majority - 2 of 3 - to change metadata. With two nodes down the cluster cannot elect leaders or create topics. (Production clusters avoid this by running the controllers separately.)

Exercise 3.4 · The broker comes back

Task: start broker 2 again. Check the ISR of `orders`, the `strict-payments` message from the note above, and who leads each partition. Then put leadership back where it belongs.

✓ SOLUTION

```
docker start kafka2
```

POWERSHELL

Wait until `docker compose ps` shows kafka2 as *healthy*, then:

```
kafka-topics.sh --bootstrap-server kafka1:9092 --describe --topic orders
```

KAFKA1 SHELL

```
Topic: orders TopicId: ywkgJzfISRGp0UN2q1Y88A PartitionCount: 3 ReplicationFactor: 3 Configs:
min.insync.replicas=2
Topic: orders Partition: 0 Leader: 3 Replicas: 3,1,2 Isr: 1,2,3 Elr: LastKnownElr:
Topic: orders Partition: 1 Leader: 1 Replicas: 1,2,3 Isr: 1,2,3 Elr: LastKnownElr:
Topic: orders Partition: 2 Leader: 3 Replicas: 2,3,1 Isr: 1,2,3 Elr: LastKnownElr:
```

OUTPUT

Broker 2 copied what it missed (including grace's order) from the leaders and rejoined every ISR. The `strict-payments` message is now committed:

```
kafka-get-offsets.sh --bootstrap-server kafka1:9092 --topic strict-payments
kafka-console-consumer.sh --bootstrap-server kafka1:9092 --topic strict-payments --from-beginning --max-messages 1
```

KAFKA1 SHELL

```
strict-payments:0:1
payment-1
Processed a total of 1 messages
```

OUTPUT

But partition 2 is still led by broker 3, although its preferred leader (first in `Replicas`) is broker 2. Broker 3 now leads two partitions and broker 2 none. Rebalance leadership with a **preferred leader election**:

```
kafka-leader-election.sh --bootstrap-server kafka1:9092 --election-type preferred --all-topic-partitions
```

KAFKA1 SHELL

```
Successfully completed leader election (PREFERRED) for partitions __consumer_offsets-47, __consumer_offsets-48, __consumer_offsets-16, pinned-1, audit-log-5, __consumer_offsets-14, payments-5, __consumer_offsets-11, payments-0, ..., orders-2, ...
```

OUTPUT

```
kafka-topics.sh --bootstrap-server kafka1:9092 --describe --topic orders
```

KAFKA1 SHELL

OUTPUT

```
Topic: orders Partition: 0 Leader: 3 Replicas: 3,1,2 Isr: 1,2,3
Topic: orders Partition: 1 Leader: 1 Replicas: 1,2,3 Isr: 1,2,3
Topic: orders Partition: 2 Leader: 2 Replicas: 2,3,1 Isr: 1,2,3
```

Each broker leads one `orders` partition again. Brokers do this on their own every few minutes (`auto.leader.rebalance.enable=true`, checked every `leader.imbalance.check.interval.seconds=300`); the command just does it now. To target one partition instead of all, use `--topic orders --partition 2`.

Exercise 3.5 · Increase the replication factor

Task: create `legacy-events` with 2 partitions and replication factor 1 (a common mistake). Give it replication factor 3 without losing data.

✓ SOLUTION

KAFKA1 SHELL

```
kafka-topics.sh --bootstrap-server kafka1:9092 --create --topic legacy-events --partitions 2 --
replication-factor 1
kafka-topics.sh --bootstrap-server kafka1:9092 --describe --topic legacy-events
```

OUTPUT

```
Created topic legacy-events.
Topic: legacy-events TopicId: amXgW7CNSq2ArG2NHX68eg PartitionCount: 2 ReplicationFactor: 1 Configs:
min.insync.replicas=2
  Topic: legacy-events Partition: 0 Leader: 3 Replicas: 3 Isr: 3
  Topic: legacy-events Partition: 1 Leader: 1 Replicas: 1 Isr: 1
```

There is no `--replication-factor` option for `--alter`. Instead you describe the new replica lists in a JSON file and hand it to `kafka-reassign-partitions.sh`. The lab has one ready:

KAFKA1 SHELL

```
cat /lab/config/increase-rf.json
```

OUTPUT

```
{
  "version": 1,
  "partitions": [
    { "topic": "legacy-events", "partition": 0, "replicas": [1, 2, 3] },
    { "topic": "legacy-events", "partition": 1, "replicas": [2, 3, 1] }
  ]
}
```

Execute - Kafka starts copying the partitions to the new brokers in the background:

KAFKA1 SHELL

```
kafka-reassign-partitions.sh --bootstrap-server kafka1:9092 \
--reassignment-json-file /lab/config/increase-rf.json --execute
```

OUTPUT

Current partition replica assignment

```
{"version":1,"partitions":[{"topic":"legacy-events","partition":0,"replicas":[3],"log_dirs":
["/var/lib/kafka/data"]},{ "topic":"legacy-events","partition":1,"replicas":[1],"log_dirs":
["/var/lib/kafka/data"]}]}
```

Save this to use as the `--reassignment-json-file` option during rollback
Successfully started partition reassignments for legacy-events-0,legacy-events-1

Verify - run until every partition says *completed*:

KAFKA1 SHELL

```
kafka-reassign-partitions.sh --bootstrap-server kafka1:9092 \
  --reassignment-json-file /lab/config/increase-rf.json --verify
kafka-topics.sh --bootstrap-server kafka1:9092 --describe --topic legacy-events
```

OUTPUT

```
Status of partition reassignment:
Reassignment of partition legacy-events-0 is completed.
Reassignment of partition legacy-events-1 is completed.

Clearing broker-level throttles on brokers 1,2,3
Clearing topic-level throttles on topic legacy-events
Topic: legacy-events TopicId: amXgW7CNSq2ArG2NHX68eg PartitionCount: 2 ReplicationFactor: 3 Configs:
min.insync.replicas=2
  Topic: legacy-events Partition: 0 Leader: 3 Replicas: 1,2,3 Isr: 1,2,3
  Topic: legacy-events Partition: 1 Leader: 1 Replicas: 2,3,1 Isr: 1,2,3
```

The leaders did not change during the move (broker 3 still leads partition 0), so clients were never interrupted. A preferred leader election (Exercise 3.4) would now hand partition 0 to broker 1 and partition 1 to broker 2.

LET KAFKA PROPOSE A PLAN

For big moves - emptying a broker before retiring it, for example - `--generate` writes the JSON for you. Here it proposes moving `pinned` onto brokers 1 and 3 only:

KAFKA1 SHELL

```
echo '{"version":1,"topics":[{"topic":"pinned"}]}' > /tmp/topics.json
kafka-reassign-partitions.sh --bootstrap-server kafka1:9092 \
  --topics-to-move-json-file /tmp/topics.json --broker-list 1,3 --generate
```

OUTPUT

```
Current partition replica assignment
{"version":1,"partitions":[{"topic":"pinned","partition":0,"replicas":[1,2],...},
{"topic":"pinned","partition":1,"replicas":[2,3],...}, {"topic":"pinned","partition":2,"replicas":[3,1],...}]}

Proposed partition reassignment configuration
{"version":1,"partitions":[{"topic":"pinned","partition":0,"replicas":[3,1],"log_dirs":["any","any"]},
{"topic":"pinned","partition":1,"replicas":[1,3],"log_dirs":["any","any"]},
{"topic":"pinned","partition":2,"replicas":[1,3],"log_dirs":["any","any"]}]}

```

Save the proposal to a file and run it with `--execute`, as above. On a busy cluster, add `--throttle 50000000` (bytes/second) so the copying does not starve your clients.

Check yourself

1. A partition shows `Replicas: 2,3,1 Isr: 3,1`. Which broker is the preferred leader, and is it serving the partition?
2. With `min.insync.replicas=2` and replication factor 3, how many brokers can fail before `acks=all` writes stop?
3. Why did `docker stop kafka2` not lose any committed data?

✓ ANSWERS

1. Broker 2 is the preferred leader (first in `Replicas`). It is not in the ISR, so it cannot be the leader right now; broker 3 or 1 is.
2. One. With one broker down, 2 copies are still in sync (enough). With two down, only 1 is left and `acks=all` writes fail with `NotEnoughReplicasException`.
3. Every committed message was already on at least two brokers (`min.insync.replicas=2`), and the controller only elects leaders from the in-sync replicas.

Lab 4 · Segments, Retention and Compaction

Goal: see that a partition is not one file but a chain of **segments**, watch new segments being created, and watch old data leave through **retention**, **record deletion** and **compaction**.

Background: what a partition looks like on disk

A partition directory holds one or more **segments**. Each segment is a set of files that share a name - the **base offset**, the first offset in that segment, padded to 20 digits:

File	Holds
<code>000000000000000004992.log</code>	The messages themselves, in record batches (Lab 8)
<code>000000000000000004992.index</code>	Offset index: offset -> byte position in the <code>.log</code> (Lab 9)
<code>000000000000000004992.timeindex</code>	Time index: timestamp -> offset (Lab 9)
<code>000000000000000004992.snapshot</code>	Producer state (producer IDs and sequence numbers) at this point, for idempotence
<code>leader-epoch-checkpoint</code>	Which leader epoch started at which offset - used to truncate diverged replicas
<code>partition.metadata</code>	The topic ID this directory belongs to

Only the newest segment - the **active segment** - is written to. A new segment is **rolled** when the active one reaches `segment.bytes` (default 1 GB) or gets older than `segment.ms` (default 7 days). Retention and compaction only ever touch **closed** segments.

Exercise 4.1 · Watch segments roll

Task: create `segments-demo` with one partition on broker 1 and `segment.bytes=1048576` (1 MB, the minimum). Write 20,000 messages of 200 bytes and list the partition directory before and after.

✓ SOLUTION

`--replica-assignment 1` means *one partition, one replica, on broker 1* - so we know exactly where the files are:

```
kafka-topics.sh --bootstrap-server kafka1:9092 --create --topic segments-demo \
  --replica-assignment 1 --config segment.bytes=1048576
ls -l /var/lib/kafka/data/segments-demo-0/
```

KAFKA1 SHELL

```
Created topic segments-demo.
-rw-r--r-- 1 appuser appuser 10485760 Sep 27 11:57 00000000000000000000.index
-rw-r--r-- 1 appuser appuser      0 Sep 27 11:57 00000000000000000000.log
-rw-r--r-- 1 appuser appuser 10485756 Sep 27 11:57 00000000000000000000.timeindex
-rw-r--r-- 1 appuser appuser      8 Sep 27 11:57 leader-epoch-checkpoint
-rw-r--r-- 1 appuser appuser    43 Sep 27 11:57 partition.metadata
```

OUTPUT

An empty partition already has one segment: an empty `.log`, and index files **pre-allocated** at 10 MB (`segment.index.bytes`) so they can be memory-mapped at a fixed size.

`kafka-producer-perf-test.sh` is the quickest way to write a lot of data. `--throughput -1` means "as fast as possible":

KAFKA1 SHELL

```
kafka-producer-perf-test.sh --bootstrap-server kafka1:9092 --topic segments-demo \
  --num-records 20000 --record-size 200 --throughput -1 \
  --command-property acks=1 linger.ms=5
```

OUTPUT

```
20000 records sent, 14970.059880 records/sec (2.86 MB/sec), 202.15 ms avg latency, 593.00 ms max latency, 208
ms 50th, 401 ms 95th, 419 ms 99th, 430 ms 99.9th.
```

KAFKA1 SHELL

```
ls -l /var/lib/kafka/data/segments-demo-0/
```

OUTPUT

```
-rw-r--r-- 1 appuser appuser      504 Sep 27 11:58 00000000000000000000.index
-rw-r--r-- 1 appuser appuser 1048128 Sep 27 11:58 00000000000000000000.log
-rw-r--r-- 1 appuser appuser      684 Sep 27 11:58 00000000000000000000.timeindex
-rw-r--r-- 1 appuser appuser      504 Sep 27 11:58 00000000000000004992.index
-rw-r--r-- 1 appuser appuser 1048128 Sep 27 11:58 00000000000000004992.log
-rw-r--r-- 1 appuser appuser      10 Sep 27 11:58 00000000000000004992.snapshot
-rw-r--r-- 1 appuser appuser      660 Sep 27 11:58 00000000000000004992.timeindex
-rw-r--r-- 1 appuser appuser      504 Sep 27 11:58 00000000000000009984.index
-rw-r--r-- 1 appuser appuser 1048128 Sep 27 11:58 00000000000000009984.log
-rw-r--r-- 1 appuser appuser      10 Sep 27 11:58 00000000000000009984.snapshot
-rw-r--r-- 1 appuser appuser      624 Sep 27 11:58 00000000000000009984.timeindex
-rw-r--r-- 1 appuser appuser      504 Sep 27 11:58 000000000000000014976.index
-rw-r--r-- 1 appuser appuser 1048128 Sep 27 11:58 000000000000000014976.log
-rw-r--r-- 1 appuser appuser      10 Sep 27 11:58 000000000000000014976.snapshot
-rw-r--r-- 1 appuser appuser      600 Sep 27 11:58 000000000000000014976.timeindex
-rw-r--r-- 1 appuser appuser 10485760 Sep 27 11:58 000000000000000019968.index
-rw-r--r-- 1 appuser appuser      6749 Sep 27 11:58 000000000000000019968.log
-rw-r--r-- 1 appuser appuser      10 Sep 27 11:58 000000000000000019968.snapshot
-rw-r--r-- 1 appuser appuser 10485756 Sep 27 11:58 000000000000000019968.timeindex
-rw-r--r-- 1 appuser appuser       8 Sep 27 11:57 leader-epoch-checkpoint
-rw-r--r-- 1 appuser appuser      43 Sep 27 11:57 partition.metadata
```

Read it carefully:

- **Five segments.** Each closed `.log` stopped just under 1 MB (1,048,128 bytes): the next batch would not have fitted, so Kafka rolled a new segment.
- **The names are offsets.** The second segment starts at offset 4992, so the first one holds offsets 0-4991: 4,992 messages of ~210 bytes each on disk.
- **Closed segments have small indexes.** When a segment is rolled, its index files are trimmed to their real size (504 bytes = 63 entries of 8 bytes). Only the **active** segment (19968) still has the pre-allocated 10 MB index.
- `.snapshot` files appear when a segment rolls; they record producer state so that idempotent producers can be checked after a restart.

Now check the offsets and the total size:

KAFKA1 SHELL

```
kafka-get-offsets.sh --bootstrap-server kafka1:9092 --topic segments-demo --time -2
kafka-get-offsets.sh --bootstrap-server kafka1:9092 --topic segments-demo --time -1
kafka-log-dirs.sh --bootstrap-server kafka1:9092 --describe --topic-list segments-demo --broker-list 1 |
tail -1
```

OUTPUT

```
segments-demo:0:0
segments-demo:0:20000
{"brokers":[{"broker":1,"logDirs":[{"partitions":[{"partition":"segments-demo-0","size":4199261,"offsetLag":0,"isFuture":false}], "error":null,"logDir":"/var/lib/kafka/data"}]}], "version":1
}
```

`--time -2` is the **log start offset** (earliest), `--time -1` the **log end offset** (latest). The partition holds offsets 0-19999, about 4.2 MB.

ROLLING BY TIME

`segment.ms` rolls a segment when its *first* message is older than the limit - but only when the next message arrives. A topic that receives nothing never rolls. `segment.jitter.ms` adds a random delay so that thousands of partitions do not all roll at the same moment.

Exercise 4.2 · Retention by size

Task: limit `segments-demo` to about 2 MB per partition with `retention.bytes`. Watch what happens on disk over the next two minutes, and find the new earliest offset.

✓ SOLUTION

```
kafka-configs.sh --bootstrap-server kafka1:9092 --alter --entity-type topics --entity-name segments-demo \
  --add-config retention.bytes=2097152
```

OUTPUT

Completed updating config for topic segments-demo.

Wait 30-40 seconds (the lab cluster checks retention every 30 s; the default is every 5 minutes):

```
ls -l /var/lib/kafka/data/segments-demo-0/ | grep -E "log|deleted"
```

OUTPUT

```
-rw-r--r-- 1 appuser appuser      504 Sep 27 11:58 00000000000000000000.index.deleted
-rw-r--r-- 1 appuser appuser 1048128 Sep 27 11:58 00000000000000000000.log.deleted
-rw-r--r-- 1 appuser appuser    684 Sep 27 11:58 00000000000000000000.timeindex.deleted
-rw-r--r-- 1 appuser appuser    504 Sep 27 11:58 000000000000000004992.index.deleted
-rw-r--r-- 1 appuser appuser 1048128 Sep 27 11:58 000000000000000004992.log.deleted
-rw-r--r-- 1 appuser appuser    10 Sep 27 11:58 000000000000000004992.snapshot.deleted
-rw-r--r-- 1 appuser appuser    660 Sep 27 11:58 000000000000000004992.timeindex.deleted
-rw-r--r-- 1 appuser appuser 1048128 Sep 27 11:58 000000000000000009984.log
-rw-r--r-- 1 appuser appuser 1048128 Sep 27 11:58 0000000000000000014976.log
-rw-r--r-- 1 appuser appuser    6749 Sep 27 11:58 000000000000000019968.log
```

The broker log explains the decision:

```
grep -h "segments-demo-0.*Deleting segment" /opt/kafka/logs/server.log*
```

OUTPUT

```
[2026-09-27 11:59:04,025] INFO [UnifiedLog partition=segments-demo-0, dir=/var/lib/kafka/data] Deleting
segment LogSegment(baseOffset=0, size=1048128, ...) due to log retention size 2097152 breach. Log size after
deletion will be 3151133.
[2026-09-27 11:59:04,025] INFO [UnifiedLog partition=segments-demo-0, dir=/var/lib/kafka/data] Deleting
segment LogSegment(baseOffset=4992, size=1048128, ...) due to log retention size 2097152 breach. Log size
after deletion will be 2103005.
```

```
kafka-get-offsets.sh --bootstrap-server kafka1:9092 --topic segments-demo --time -2
```

OUTPUT

segments-demo:0:9984

About a minute later the `.deleted` files are gone:

```
ls -l /var/lib/kafka/data/segments-demo-0/ | grep -E "\.log"
```

KAFKA1 SHELL

```
-rw-r--r-- 1 appuser appuser 1048128 Sep 27 11:58 00000000000000009984.log
-rw-r--r-- 1 appuser appuser 1048128 Sep 27 11:58 000000000000000014976.log
-rw-r--r-- 1 appuser appuser 6749 Sep 27 11:58 000000000000000019968.log
```

OUTPUT

What happened, step by step:

1. The retention check found the partition at 4.2 MB, over the 2 MB limit.
2. It deleted **whole segments, oldest first**, as long as the partition stayed at or above the limit afterwards (4.2 -> 3.15 -> 2.10 MB). Deleting a third would have taken it below 2 MB, so it stopped. Retention never deletes part of a segment.
3. The **log start offset** moved to 9984, the base offset of the oldest remaining segment. A consumer asking for offset 5000 now gets an "offset out of range" and follows its `auto.offset.reset` setting.
4. The files were first renamed to `.deleted`, then removed after `file.delete.delay.ms` (60 s), so that any reader still using them can finish.

RETENTION BY TIME

`retention.ms` works the same way: a closed segment is deleted when its **newest** message is older than `retention.ms`. Data therefore lives *at least* as long as the retention period, and up to one segment's lifetime longer. Setting both `retention.ms` and `retention.bytes` deletes a segment when either limit is exceeded.

Exercise 4.3 · Delete records up to an offset

Task: a bad batch of messages sits at the start of `segments-demo`. Delete everything before offset 12000 - right now, without waiting for retention.

✓ SOLUTION

`kafka-delete-records.sh` takes a JSON file listing, per partition, the first offset to **keep**:

```
echo '{"partitions":[{"topic":"segments-demo","partition":0,"offset":12000}],"version":1}' >
/tmp/delete.json
kafka-delete-records.sh --bootstrap-server kafka1:9092 --offset-json-file /tmp/delete.json
```

KAFKA1 SHELL

```
Executing records delete operation
Records delete operation completed:
partition: segments-demo-0 low_watermark: 12000
```

OUTPUT

```
kafka-get-offsets.sh --bootstrap-server kafka1:9092 --topic segments-demo --time -2
ls -l /var/lib/kafka/data/segments-demo-0/ | grep -E "\.log"
```

KAFKA1 SHELL

```
segments-demo:0:12000
-rw-r--r-- 1 appuser appuser 1048128 Sep 27 11:58 00000000000000009984.log
-rw-r--r-- 1 appuser appuser 1048128 Sep 27 11:58 000000000000000014976.log
-rw-r--r-- 1 appuser appuser 6749 Sep 27 11:58 000000000000000019968.log
```

OUTPUT

The log start offset (the *low watermark*) is now 12000, although segment `9984` is still on disk: offset 12000 is in the middle of it. Offsets 9984-11999 are logically deleted - no consumer can read them - and the file goes away once retention finds a segment entirely below the log start offset. Offsets are never reused: the next message is still offset 20000.

Exercise 4.4 · Log compaction

Task: create a compacted topic `customer-profile` that keeps only the latest profile per customer. Write several updates for the same customers and a delete, force a segment roll, and compare what a consumer sees before and after compaction.

✓ SOLUTION

Compaction is switched on with `cleanup.policy=compact`. To see it in minutes rather than days, the lab also rolls segments every 10 s (`segment.ms`), cleans as soon as 1 % of the log is "dirty" (`min.cleanable.dirty.ratio`), and keeps delete markers for only 10 s (`delete.retention.ms`):

```
kafka-topics.sh --bootstrap-server kafka1:9092 --create --topic customer-profile --replica-assignment 1 \
--config cleanup.policy=compact --config segment.ms=10000 \
--config min.cleanable.dirty.ratio=0.01 --config delete.retention.ms=10000
```

Write six updates. `null.marker=NULL` turns the text `NULL` into a real null value - a **tombstone**, which means "delete this key":

```
printf 'alice:{"city":"Chennai","tier":"silver"}\nbob:{"city":"Pune","tier":"gold"}\nalice:{"city":"Chennai","tier":"gold"}\ncarol:{"city":"Delhi","tier":"silver"}\nalice:{"city":"Bengaluru","tier":"gold"}\nbob:NULL\n' \
| kafka-console-producer.sh --bootstrap-server kafka1:9092 --topic customer-profile \
--reader-property parse.key=true --reader-property key.separator=: --reader-property null.marker=NULL
```

```
kafka-console-consumer.sh --bootstrap-server kafka1:9092 --topic customer-profile --from-beginning --max-messages 6 \
--formatter-property print.offset=true --formatter-property print.key=true
```

```
Offset:0  alice  {"city":"Chennai","tier":"silver"}
Offset:1  bob    {"city":"Pune","tier":"gold"}
Offset:2  alice  {"city":"Chennai","tier":"gold"}
Offset:3  carol  {"city":"Delhi","tier":"silver"}
Offset:4  alice  {"city":"Bengaluru","tier":"gold"}
Offset:5  bob    null
Processed a total of 6 messages
```

Compaction never touches the active segment. Wait more than 10 s and write one more message, so that the segment holding offsets 0-5 is closed:

```
echo 'dave:{"city":"Mumbai","tier":"silver"}' | kafka-console-producer.sh --bootstrap-server kafka1:9092 \
--topic customer-profile --reader-property parse.key=true --reader-property key.separator=:
```

Within about 30 seconds the **log cleaner** runs:

KAFKA1 SHELL

```
ls -l /var/lib/kafka/data/customer-profile-0/ | grep -E "\.log"
grep -h "customer-profile" /opt/kafka/logs/log-cleaner.log | tail -2
```

OUTPUT

```
-rw-r--r-- 1 appuser appuser      167 Sep 27 12:00 00000000000000000000.log
-rw-r--r-- 1 appuser appuser      290 Sep 27 12:00 00000000000000000000.log.deleted
-rw-r--r-- 1 appuser appuser      105 Sep 27 12:01 000000000000000000006.log
[2026-09-27 12:01:34,426] INFO Cleaner 0: Swapping in cleaned segment LogSegment(baseOffset=0, size=167, ...)
for segment(s) [LogSegment(baseOffset=0, size=290, ...)] in log Log(dir=/var/lib/kafka/data/customer-profile-
0, ...)
Log cleaner thread 0 cleaned log customer-profile-0 (dirty section = [0, 6])
```

The cleaner wrote a new, smaller copy of segment 0 (290 -> 167 bytes) and swapped it in. Read the topic again:

KAFKA1 SHELL

```
kafka-console-consumer.sh --bootstrap-server kafka1:9092 --topic customer-profile --from-beginning --
timeout-ms 6000 \
--formatter-property print.offset=true --formatter-property print.key=true
```

OUTPUT

```
Offset:3    carol  {"city":"Delhi","tier":"silver"}
Offset:4    alice  {"city":"Bengaluru","tier":"gold"}
Offset:5    bob    null
Offset:6    dave   {"city":"Mumbai","tier":"silver"}
Processed a total of 4 messages
```

- alice's two older profiles (offsets 0 and 2) are gone; only her **latest** value remains.
- bob's first value (offset 1) is gone. His **tombstone** (offset 5) stays for `delete.retention.ms`, so that consumers who are behind still learn that bob was deleted. A later cleaning removes it too.
- **Offsets do not change.** The surviving messages keep 3, 4, 5 - the log now has gaps, and consumers simply skip them.

WHERE COMPACTION IS USED

Compacted topics are the "table" side of Kafka: the latest state per key, kept forever. Kafka itself keeps consumer offsets in the compacted topic `__consumer_offsets` (Lab 7). Kafka Connect keeps its configs, offsets and status the same way, and Kafka Streams keeps its state-store changelogs so.

Check yourself

1. The newest file in a partition directory is `00000000000000734112.log`. What do you know from the name alone?
2. `retention.ms` is 1 day and `segment.ms` is 7 days on a quiet topic. How long can a message live?
3. Why must the active segment never be compacted or deleted?

✓ ANSWERS

1. It is the active segment, and its first message is offset 734112. Every older segment ends at 734111 or below.
2. Up to about 8 days. Retention only deletes closed segments, and a segment only closes after 7 days (or 1 GB); then its newest message must be over 1 day old.
3. It is still being written. Retention and the cleaner work on immutable, closed segments, so they never race with the producer's appends.

Lab 5 · Producers: Publishing Events

Goal: write data with `kafka-console-producer.sh` in every useful way - plain lines, keys, headers, files, config files - and measure how `acks` and compression change throughput and disk usage with `kafka-producer-perf-test.sh`.

OPTION NAMES CHANGED IN KAFKA 4.X

Older books and blog posts use options that are now deprecated. This book uses the current ones:

Kafka 4.x (use these)	Deprecated	Purpose
<code>--reader-property</code>	<code>--property</code>	How the console producer parses each input line (keys, headers ...)
<code>--command-property</code>	<code>--producer-property</code>	A producer setting, such as <code>acks=all</code>
<code>--command-config</code>	<code>--producer.config</code>	A file of producer settings

Exercise 5.1 · Your first messages

Task: start the console producer on `audit-log`, type three lines, stop it, and read them back.

✓ SOLUTION

```
kafka-console-producer.sh --bootstrap-server kafka1:9092 --topic audit-log
```

KAFKA1 SHELL

The producer shows a `>` prompt. Every line you type and send with **Enter** is one message. Stop with **Ctrl+C**:

```
>user alice logged in
>user bob logged in
>user alice logged out
>^C
```

OUTPUT

Read them back in a second window (`docker exec -it kafka1 bash`):

```
kafka-console-consumer.sh --bootstrap-server kafka1:9092 --topic audit-log --from-beginning --max-
messages 3 \
  --formatter-property print.partition=true
```

KAFKA1 SHELL

```
Partition:1 user alice logged in
Partition:1 user bob logged in
Partition:1 user alice logged out
Processed a total of 3 messages
```

OUTPUT

All three went to the same partition, although they have no key. Since Kafka 2.4 the producer's default partitioner is **sticky**: messages without a key fill one batch for one partition, and only move to another partition when the batch is sent. Over time the spread is even; over a few lines, they stick together.

PIPING INSTEAD OF TYPING

Anything that writes lines to standard input works - which is how this book captured its outputs:

KAFKA1 SHELL

```
printf 'user alice logged in\nuser bob logged in\n' | kafka-console-producer.sh --bootstrap-server
kafka1:9092 --topic audit-log
kafka-console-producer.sh --bootstrap-server kafka1:9092 --topic orders < /lab/data/orders.txt
```

Exercise 5.2 · Keys, and a surprise

Task: create `key-skew` (3 partitions). Send two messages each for keys `cust-1`, `cust-2` and `cust-3`, then count messages per partition.

✓ SOLUTION

KAFKA1 SHELL

```
kafka-topics.sh --bootstrap-server kafka1:9092 --create --topic key-skew --partitions 3
printf 'cust-1:a\ncust-2:b\ncust-3:c\ncust-1:d\ncust-2:e\ncust-3:f\n' | kafka-console-producer.sh \
--bootstrap-server kafka1:9092 --topic key-skew \
--reader-property parse.key=true --reader-property key.separator=:
kafka-get-offsets.sh --bootstrap-server kafka1:9092 --topic key-skew
```

OUTPUT

```
key-skew:0:0
key-skew:1:6
key-skew:2:0
```

All six messages are in partition 1. Nothing is broken: `murmur2("cust-1")`, `murmur2("cust-2")` and `murmur2("cust-3")` all happen to give remainder 1 when divided by 3. With only three keys, collisions like this are common.

This is **key skew**: one partition (and so one broker, and one consumer in a group) does all the work while the others sit idle. In real systems it comes from a few very busy keys (one huge customer) or from too few distinct keys. Keys should have many distinct values - customer IDs, device IDs, account numbers - not a handful of categories.

The reader properties of the console producer:

--reader-property	Default	Meaning
<code>parse.key</code>	<code>false</code>	Split each line into key and value
<code>key.separator</code>	<code>tab</code>	Where to split
<code>parse.headers</code>	<code>false</code>	Read headers at the start of each line
<code>headers.delimiter</code>	<code>tab</code>	Separates the headers from the rest of the line
<code>headers.separator</code>	<code>,</code>	Separates one header from the next
<code>headers.key.separator</code>	<code>:</code>	Separates a header's name from its value
<code>null.marker</code>	<code>(none)</code>	Text to turn into a null key or value - used for tombstones in Lab 4
<code>ignore.error</code>	<code>false</code>	Skip lines that do not contain the key separator instead of stopping

Exercise 5.3 · Headers

Task: send an order for key `grace` with two headers, `source=web` and `trace=abc123`, and read it back showing the headers.

✓ SOLUTION

With `parse.headers=true` each line is `headers<TAB>key<TAB>value`, and the headers are written `name:value,name:value`:

```
printf 'source:web,trace:abc123\tgrace\t{"order":2012,"item":"notebook","amount":12}\n' \
| kafka-console-producer.sh --bootstrap-server kafka1:9092 --topic orders \
--reader-property parse.key=true --reader-property parse.headers=true
```

KAFKA1 SHELL

`grace` hashes to partition 0 of `orders`. Read the last two messages there:

```
kafka-console-consumer.sh --bootstrap-server kafka1:9092 --topic orders --partition 0 --offset 4 --max-
messages 2 \
--formatter-property print.headers=true --formatter-property print.key=true --formatter-property
print.offset=true
```

KAFKA1 SHELL

```
Offset:4    NO_HEADERS  grace  {"order":2011,"item":"pen","amount":3}
Offset:5    source:web,trace:abc123 grace  {"order":2012,"item":"notebook","amount":12}
Processed a total of 2 messages
```

OUTPUT

Headers carry metadata *about* the message - where it came from, a trace ID, a schema version - without touching the value. Kafka never looks inside them; they are for your applications.

A COMMON MISTAKE

Writing headers as `source=web` fails with `No header key separator found in pair 'source=web'`. The separator between a header's name and value is `:` unless you change `headers.key.separator`.

Exercise 5.4 · Producer settings from a file

Task: send the 10 sample orders to `payments` using the settings in `/lab/config/producer.properties`. What does each setting do? Then check which compression was actually used.

✓ SOLUTION

```
grep -v '^#' /lab/config/producer.properties | grep .
```

KAFKA1 SHELL

```
client.id=lab-producer
acks=all
enable.idempotence=true
linger.ms=20
batch.size=32768
compression.type=snappy
delivery.timeout.ms=120000
```

OUTPUT

Setting	Meaning
<code>client.id</code>	A name for this producer in broker logs, metrics and quotas
<code>acks</code>	<code>0</code> = do not wait; <code>1</code> = wait for the leader; <code>all</code> = wait for every in-sync replica (the default since Kafka 3.0)
<code>enable.idempotence</code>	The broker drops duplicates caused by retries, using producer ID + sequence number (default <code>true</code>)
<code>linger.ms</code>	Wait up to this long to fill a batch before sending (default 5 ms in Kafka 4.x)
<code>batch.size</code>	Maximum bytes per batch, per partition (default 16384)
<code>compression.type</code>	<code>none</code> , <code>gzip</code> , <code>snappy</code> , <code>lz4</code> or <code>zstd</code> - a whole batch is compressed together
<code>delivery.timeout.ms</code>	Total time <code>send()</code> may take, including retries, before it reports failure

```
kafka-console-producer.sh --bootstrap-server kafka1:9092 --topic payments \
--command-config /lab/config/producer.properties \
--reader-property parse.key=true --reader-property key.separator=: < /lab/data/orders.txt
kafka-get-offsets.sh --bootstrap-server kafka1:9092 --topic payments
```

KAFKA1 SHELL

```
payments:0:1
payments:1:1
payments:2:3
payments:3:3
payments:4:2
payments:5:0
```

OUTPUT

Ten messages across six partitions, spread by key. Now look at one batch on disk with `kafka-dump-log.sh` (explained fully in Lab 8):

```
kafka-dump-log.sh --files /var/lib/kafka/data/payments-2/00000000000000000000.log | grep -o 'count: [0-9]*\\|compresscodec: [a-z0-9]*'
```

KAFKA1 SHELL

```
count: 3
compresscodec: none
```

OUTPUT

Not compressed - although the file says `lz4`.

CONSOLE PRODUCER GOTCHA: `--COMPRESSION-CODEC` WINS

`kafka-console-producer.sh` has its own `--compression-codec` option, which defaults to `none` and **overrides** `compression.type` from `--command-config` and `--command-property`. Proof - one message each way into a fresh topic:

```
kafka-topics.sh --bootstrap-server kafka1:9092 --create --topic codec-check --replica-assignment 1
echo 'x1' | kafka-console-producer.sh --bootstrap-server kafka1:9092 --topic codec-check --command-
property compression.type=lz4
echo 'x2' | kafka-console-producer.sh --bootstrap-server kafka1:9092 --topic codec-check --compression-
codec lz4
kafka-dump-log.sh --files /var/lib/kafka/data/codec-check-0/00000000000000000000.log | grep -o
'baseOffset: [0-9]*\\|compresscodec: [a-z0-9]*'
```

KAFKA1 SHELL

OUTPUT

```
baseOffset: 0
compresscodec: none
baseOffset: 1
compresscodec: lz4
```

With the console producer, use `--compression-codec`. Real applications, and `kafka-producer-perf-test.sh`, honour `compression.type`.

Exercise 5.5 · What `acks` costs

Task: create `perf-test` (3 partitions, replication factor 3). Send 100,000 messages of 500 bytes with `acks=0`, `acks=1` and `acks=all`, and compare throughput and latency.

✓ SOLUTION

KAFKA1 SHELL

```
kafka-topics.sh --bootstrap-server kafka1:9092 --create --topic perf-test --partitions 3
for a in 0 1 all; do
  kafka-producer-perf-test.sh --bootstrap-server kafka1:9092 --topic perf-test \
    --num-records 100000 --record-size 500 --throughput -1 \
    --command-property acks=$a linger.ms=10 batch.size=65536
done
```

OUTPUT

```
100000 records sent, 38431.975404 records/sec (18.33 MB/sec), 23.20 ms avg latency, 766.00 ms max latency, 13
ms 50th, 71 ms 95th, 84 ms 99th, 108 ms 99.9th.
100000 records sent, 38022.813688 records/sec (18.13 MB/sec), 178.93 ms avg latency, 680.00 ms max latency,
169 ms 50th, 356 ms 95th, 391 ms 99th, 405 ms 99.9th.
100000 records sent, 31328.320802 records/sec (14.94 MB/sec), 445.89 ms avg latency, 979.00 ms max latency,
396 ms 50th, 882 ms 95th, 954 ms 99th, 978 ms 99.9th.
```

acks	Records/s	Avg latency	99th pct	Durability
<code>0</code>	38,432	23 ms	84 ms	None - the producer does not even learn about failures
<code>1</code>	38,023	179 ms	391 ms	Lost if the leader dies before followers copy the message
<code>all</code>	31,328	446 ms	954 ms	Survives any failure that leaves one in-sync copy

`acks=all` waits for the slowest in-sync follower, so latency grows most. On a laptop the three "brokers" share one disk and CPU, which exaggerates the gap; on real hardware it is smaller. The rule stays: **`acks=all` for data you cannot lose** - it is the default for a reason. (Your numbers will differ from these.)

Exercise 5.6 · What compression saves

Task: send the same 50,000 JSON orders (`/lab/data/payloads.txt`) to four single-partition topics with `none`, `gzip`, `lz4` and `zstd`. Compare the size on disk.

✓ SOLUTION

KAFKA1 SHELL

```
for z in none gzip lz4 zstd; do
  kafka-topics.sh --bootstrap-server kafka1:9092 --create --topic comp-$z --replica-assignment 1
done
for z in none gzip lz4 zstd; do
```

```
kafka-producer-perf-test.sh --bootstrap-server kafka1:9092 --topic comp-$z \
  --num-records 50000 --payload-file /lab/data/payloads.txt --throughput -1 \
  --command-property acks=all linger.ms=20 batch.size=65536 compression.type=$z | tail -1
done
du -b /var/lib/kafka/data/comp-*/*.log
```

OUTPUT

```
50000 records sent, 46253.469010 records/sec (4.89 MB/sec), 9.40 ms avg latency, ...
50000 records sent, 33898.305085 records/sec (3.58 MB/sec), 11.66 ms avg latency, ...
50000 records sent, 32743.942371 records/sec (3.46 MB/sec), 14.30 ms avg latency, ...
50000 records sent, 42698.548249 records/sec (4.51 MB/sec), 9.54 ms avg latency, ...
669235 /var/lib/kafka/data/comp-gzip-0/00000000000000000000.log
1240001 /var/lib/kafka/data/comp-lz4-0/00000000000000000000.log
6040650 /var/lib/kafka/data/comp-none-0/00000000000000000000.log
760843 /var/lib/kafka/data/comp-zstd-0/00000000000000000000.log
```

Codec	On disk	vs none
none	6.04 MB	100 %
lz4	1.24 MB	21 %
zstd	0.76 MB	13 %
gzip	0.67 MB	11 %

- `--payload-file` sends real JSON lines, which compress well. (The default random payload of `--record-size` hardly compresses at all.)
- The producer compresses **whole batches**, so bigger batches (higher `linger.ms` / `batch.size`) compress better.
- The broker stores the batch **as the producer compressed it** (topic `compression.type=producer`) and the consumer decompresses it. Compression saves network, disk and replication traffic at the cost of producer and consumer CPU.
- `zstd` gives near-gzip size at much lower CPU; `lz4` is the fastest. Both are common choices.

Exercise 5.7 · Keys across languages

Task: in Lab 1 (Exercise 1.4) the Python script wrote an order with key `judy`. Which partition did it go to, and which partition would a Java producer choose?

✓ SOLUTION

The Python output said `written : orders-0 @ offset 6`: partition **0**. Java producers - including `kafka-console-producer.sh` - hash keys with **murmur2**, which puts `judy` in partition **2**. (In Lab 7, judy's order from the console producer is indeed in partition 2.)

The Python, Go, .NET and JavaScript clients are built on **librdkafka**, whose default partitioner is `consistent_random` (CRC32), not murmur2. If producers in different languages write keyed messages to the same topic, the same key can land in different partitions, breaking per-key order. Fix: set `partitioner=murmur2_random` in every librdkafka-based producer.

Check yourself

1. You need `acks=all` and lz4 compression from the console producer. Which options do you pass?

2. Why can three keys all land in one partition, and why does it matter?
3. Your producer sends with `acks=all` and gets `NotEnoughReplicasException`. What is wrong, and where do you look?

✓ ANSWERS

1. `--command-property acks=all --compression-codec lz4` (the console producer's own `--compression-codec` overrides `compression.type`).
2. The partition is `hash(key) mod partitions`; with few keys, collisions are likely. It matters because one partition - one broker and one consumer - gets all the load: key skew.
3. The partition has fewer in-sync replicas than `min.insync.replicas`. Check `kafka-topics.sh --describe --under-min-isr-partitions` and look for a broker that is down or lagging (Lab 3).

Lab 6 · Consumers, Consumer Properties and Offsets

Goal: read data with `kafka-console-consumer.sh` from any point in a topic, show everything a record carries, configure a consumer from a file, and understand the different kinds of offset.

Background: the offsets you will meet

For every partition, several offsets matter:

```

offset:  0  1  2  3  4  5  6  7  8  9
+-----+-----+-----+-----+-----+
| x | x | x | x | x | x | x | x | . | . |
+-----+-----+-----+-----+-----+
      ^             ^             ^             ^
log start  committed      high      log end
offset      offset        watermark  offset
(earliest) (a group's      (last safe (next offset
              position)    + 1)      to write)
  
```

OUTPUT

Offset	Meaning	How to see it
Log start offset	The oldest offset still stored. Moves forward with retention and <code>kafka-delete-records</code> .	<code>kafka-get-offsets.sh --time -2</code>
High watermark	Messages below it are on all in-sync replicas: <i>committed</i> . Consumers only ever see messages below it.	<code>kafka-get-offsets.sh --time -1</code> (what <code>-1</code> returns)
Log end offset	The offset the next message will get, on the leader. Equal to the high watermark when all replicas are caught up.	<code>LOG-END-OFFSET</code> in <code>kafka-consumer-groups.sh</code>
Committed offset	Where a consumer group will resume: the offset of the <i>next</i> message to read, per partition. Stored in <code>__consumer_offsets</code> .	<code>CURRENT-OFFSET</code> in <code>kafka-consumer-groups.sh</code>
Lag	High watermark minus committed offset: how far the group is behind.	<code>LAG</code> in <code>kafka-consumer-groups.sh</code>

Exercise 6.1 · Latest vs beginning

Task: start a console consumer on `orders` without any options and wait 5 seconds. Then start one with `--from-beginning`. Why the difference?

✓ SOLUTION

```
kafka-console-consumer.sh --bootstrap-server kafka1:9092 --topic orders --timeout-ms 5000
```

KAFKA1 SHELL

```
Processed a total of 0 messages
```

OUTPUT

```
kafka-console-consumer.sh --bootstrap-server kafka1:9092 --topic orders --from-beginning --max-messages 3
```

KAFKA1 SHELL

OUTPUT

```
{ "order": 2001, "item": "keyboard", "amount": 45 }
{ "order": 2004, "item": "headset", "amount": 60 }
{ "order": 2007, "item": "cable", "amount": 9 }
Processed a total of 3 messages
```

A new consumer with no committed offset starts according to `auto.offset.reset`. The default is `latest`: start at the end and wait for **new** messages - that is why the first one printed nothing. `--from-beginning` sets it to `earliest`.

Useful stop conditions: `--max-messages N` exits after N messages; `--timeout-ms T` exits after T ms without a message. Without them the consumer runs until **Ctrl+C**.

THE BANNER YOU WILL SEE

Every console consumer in Kafka 4.3 first prints: *"The consumer rebalance protocol (KIP-848) is production-ready! Set group.protocol=consumer to try it out."* It is informational. Lab 7 tries the new protocol.

Exercise 6.2 · Show everything a record carries

Task: print the first four messages of `orders` with their timestamp, partition, offset, headers and key, separated by `|`.

✓ SOLUTION

KAFKA1 SHELL

```
kafka-console-consumer.sh --bootstrap-server kafka1:9092 --topic orders --from-beginning --max-messages 4 \
--formatter-property print.timestamp=true --formatter-property print.partition=true \
--formatter-property print.offset=true --formatter-property print.headers=true \
--formatter-property print.key=true --formatter-property key.separator='|'
```

OUTPUT

```
CreateTime:1790510023037 | Partition:0 | Offset:0 | NO_HEADERS | alice |
{ "order": 2001, "item": "keyboard", "amount": 45 }
CreateTime:1790510023055 | Partition:0 | Offset:1 | NO_HEADERS | alice |
{ "order": 2004, "item": "headset", "amount": 60 }
CreateTime:1790510023056 | Partition:0 | Offset:2 | NO_HEADERS | bob |
{ "order": 2007, "item": "cable", "amount": 9 }
CreateTime:1790510023056 | Partition:0 | Offset:3 | NO_HEADERS | alice | { "order": 2010, "item": "laptop
stand", "amount": 55 }
```

--formatter-property

Prints

<code>print.timestamp=true</code>	<code>CreateTime</code> (set by the producer) or <code>LogAppendTime</code> (set by the broker, if the topic says so), in epoch milliseconds
<code>print.partition=true</code> / <code>print.offset=true</code>	Where the record lives
<code>print.headers=true</code>	Headers as <code>name:value,...</code> , or <code>NO_HEADERS</code>
<code>print.key=true</code> / <code>print.value=false</code>	Show the key / hide the value
<code>key.separator</code>	Text between fields (default: tab)
<code>null.literal</code>	What to print for a null key or value (default <code>null</code>)

1790510023037 is milliseconds since 1970: 27 Sep 2026, 11:53:43 UTC.

Exercise 6.3 · Read from an exact position

Task:

1. Read 2 messages from partition 1 of `orders`, starting at offset 1.
2. Read partition 2 from its earliest offset.

✓ SOLUTION

`--offset` needs `--partition`: an offset only means something inside one partition.

KAFKA1 SHELL

```
kafka-console-consumer.sh --bootstrap-server kafka1:9092 --topic orders --partition 1 --offset 1 --max-  
messages 2 \  
  --formatter-property print.offset=true --formatter-property print.key=true
```

OUTPUT

```
Offset:1    frank    {"order":2006,"item":"dock","amount":140}  
Offset:2    dave     {"order":2008,"item":"chair","amount":320}  
Processed a total of 2 messages
```

KAFKA1 SHELL

```
kafka-console-consumer.sh --bootstrap-server kafka1:9092 --topic orders --partition 2 --offset earliest  
--max-messages 3 \  
  --formatter-property print.offset=true
```

OUTPUT

```
Offset:0    {"order":2003,"item":"mouse","amount":18}  
Offset:1    {"order":2005,"item":"webcam","amount":75}  
Offset:2    {"order":2009,"item":"desk lamp","amount":35}  
Processed a total of 3 messages
```

`--offset` accepts a number, `earliest` or `latest`. Reading a partition directly like this uses **no consumer group** and commits nothing - ideal for inspecting data or replaying one partition by hand.

Exercise 6.4 · The offsets of a topic

Task: find the earliest offset, the latest offset, and the offset of the message with the highest timestamp in each partition of `orders`.

✓ SOLUTION

KAFKA1 SHELL

```
kafka-get-offsets.sh --bootstrap-server kafka1:9092 --topic orders --time -2  
kafka-get-offsets.sh --bootstrap-server kafka1:9092 --topic orders --time -1  
kafka-get-offsets.sh --bootstrap-server kafka1:9092 --topic orders --time -3
```

OUTPUT

```
orders:0:0  
orders:1:0  
orders:2:0  
---  
orders:0:6  
orders:1:3  
orders:2:3
```

```
---
orders:0:5
orders:1:1
orders:2:2
```

<code>--time</code>	Name	Returns
<code>-2</code>	<code>earliest</code>	The log start offset
<code>-1</code>	<code>latest</code> (default)	The high watermark - the next offset to be read
<code>-3</code>	<code>max-timestamp</code>	The offset of the record with the largest timestamp
<code>-4</code>	<code>earliest-local</code>	The earliest offset still on the broker's disk (differs only with tiered storage)
<code>-5</code>	<code>latest-tiered</code>	The last offset copied to remote storage (tiered storage)
<code>-6</code>	<code>earliest-pending-upload</code>	The first offset not yet copied to remote storage (tiered storage)
<code>epoch</code> <code>ms</code>		The first offset whose timestamp is at or after that time (uses the time index - Lab 9)

The names work too: `--time earliest` is the same as `--time -2`.

Partition 0 holds offsets 0-5: six messages (alice x3, bob, and grace x2).

Exercise 6.5 · Consumer properties and committed offsets

Task: use `/lab/config/consumer.properties` to read 5 messages from `orders`. Which group did it use, and what did it commit?

✓ SOLUTION

```
grep -v '^#' /lab/config/consumer.properties | grep .
```

KAFKA1 SHELL

```
client.id=lab-consumer
group.id=orders-reporting
auto.offset.reset=earliest
enable.auto.commit=true
auto.commit.interval.ms=5000
max.poll.records=100
isolation.level=read_committed
```

OUTPUT

```
kafka-console-consumer.sh --bootstrap-server kafka1:9092 --topic orders \
  --command-config /lab/config/consumer.properties --max-messages 5 --formatter-property print.key=true
```

KAFKA1 SHELL

```
alice  {"order":2001,"item":"keyboard","amount":45}
alice  {"order":2004,"item":"headset","amount":60}
bob    {"order":2007,"item":"cable","amount":9}
alice  {"order":2010,"item":"laptop stand","amount":55}
grace  {"order":2011,"item":"pen","amount":3}
Processed a total of 5 messages
```

OUTPUT

```
kafka-consumer-groups.sh --bootstrap-server kafka1:9092 --describe --group orders-reporting
```

KAFKA1 SHELL

OUTPUT

Consumer group 'orders-reporting' has no active members.

GROUP	TOPIC	PARTITION	CURRENT-OFFSET	LOG-END-OFFSET	LAG	CONSUMER-ID	HOST	CLIENT-ID
orders-reporting	orders	0	5	6	1	-	-	-
orders-reporting	orders	1	0	3	3	-	-	-
orders-reporting	orders	2	0	3	3	-	-	-

- The group committed **5** for partition 0: it read offsets 0-4, so the *next* message it needs is offset 5. A committed offset always points at the next message, never the last one read.
- It read nothing from partitions 1 and 2 before `--max-messages 5` stopped it, so it committed their starting position, 0.
- **LAG** is how much is left: $1 + 3 + 3 = 7$ messages. Run the same command again and it resumes exactly there.

The consumer settings you will tune most:

Setting	Default	Meaning
<code>group.id</code>	(none)	The consumer group. Without it there are no commits and no partition sharing.
<code>auto.offset.reset</code>	<code>latest</code>	Where to start when the group has no committed offset (or it is out of range): <code>earliest</code> , <code>latest</code> or <code>none</code> (throw an error)
<code>enable.auto.commit</code>	<code>true</code>	Commit automatically in the background ...
<code>auto.commit.interval.ms</code>	5000	... every 5 s. Turn auto-commit off to commit only after processing.
<code>max.poll.records</code>	500	Records returned by one <code>poll()</code>
<code>max.poll.interval.ms</code>	300000	If <code>poll()</code> is not called for this long, the consumer is thrown out of the group
<code>session.timeout.ms</code>	45000	No heartbeat for this long = consumer considered dead (classic protocol)
<code>fetch.min.bytes</code> / <code>fetch.max.wait.ms</code>	1 / 500	Wait for this much data, or this long, before a fetch returns
<code>isolation.level</code>	<code>read_uncommitted</code>	<code>read_committed</code> hides messages from aborted or open transactions
<code>group.protocol</code>	<code>classic</code>	<code>consumer</code> selects the new KIP-848 rebalance protocol (Lab 7)

WHERE DO `--COMMAND-CONFIG` SETTINGS GO?

`--command-config FILE` and `--command-property key=value` pass any consumer setting. The console consumer's own options - `--group`, `--from-beginning`, `--isolation-level` - are shortcuts for some of them.

Exercise 6.6 · Several topics at once

Task: read from both `orders` and `payments` with one consumer.

✓ SOLUTION

`--include` takes a regular expression instead of `--topic`:

KAFKA1 SHELL

```
kafka-console-consumer.sh --bootstrap-server kafka1:9092 --include 'orders|payments' --from-beginning \
--max-messages 30 --timeout-ms 8000 --formatter-property print.partition=true | grep -c Partition
```

22

OUTPUT

12 messages from `orders` plus 10 from `payments` (at that point in the labs). This is the same regex subscription a Java consumer does with `subscribe(Pattern)`: topics created later that match are picked up automatically.

Check yourself

1. A group's `CURRENT-OFFSET` for a partition is 120 and `LOG-END-OFFSET` is 150. Which offset will it read next, and what is its lag?
2. A new consumer group reads a topic that holds a year of data, and the team wants only new messages. Which setting matters?
3. Retention deleted offsets 0-999. A group's committed offset is 500. What happens when it starts?

✓ ANSWERS

1. Offset 120; lag 30.
2. `auto.offset.reset=latest` (the default) - it only applies because the group has no committed offset yet.
3. Offset 500 is out of range, so the consumer applies `auto.offset.reset: earliest` jumps to 1000 (the new log start), `latest` to the end, `none` throws an error.

Lab 7 · Consumer Groups

Goal: run several consumers in one group and watch Kafka share the partitions between them, rebalance when members come and go, track lag, reset offsets, and look at the internal topic where it is all stored.

OPEN FOUR TERMINALS

This lab runs several consumers at the same time. Open four PowerShell windows in the lab folder. Terminals A, B and C run consumers; terminal D runs the admin commands (`docker exec -it kafka1 bash`).

Exercise 7.1 · One consumer, all partitions

Task: start one console consumer in group `orders-app` with client ID `app-1`. Describe the group: its members and its offsets.

✓ SOLUTION

Terminal A:

```
docker exec -it kafka1 kafka-console-consumer.sh --bootstrap-server kafka1:9092 --topic orders `
--group orders-app --command-property client.id=app-1 `
--formatter-property print.partition=true --formatter-property print.key=true
```

POWERSHELL

(In PowerShell the backtick ``` continues a line.) The consumer waits for new messages.

Terminal D:

```
kafka-consumer-groups.sh --bootstrap-server kafka1:9092 --describe --group orders-app --members --
verbose
```

KAFKA1 SHELL

GROUP	CONSUMER-ID	HOST	CLIENT-ID	#PARTITIONS	CURRENT-	OUTPUT
ASSIGNMENT						
orders-app	app-1-20cddbc5-1f06-4544-bab3-65dbc0787af2	/172.21.0.2	app-1	3	orders:0,1,2	

```
kafka-consumer-groups.sh --bootstrap-server kafka1:9092 --describe --group orders-app
```

KAFKA1 SHELL

GROUP	TOPIC	PARTITION	CURRENT-OFFSET	LOG-END-OFFSET	LAG	CONSUMER-ID	OUTPUT
HOST	CLIENT-ID						
orders-app	orders	0	6	6	0	app-1-20cddbc5-1f06-4544-bab3-65dbc0787af2	
/172.21.0.2	app-1						
orders-app	orders	1	3	3	0	app-1-20cddbc5-1f06-4544-bab3-65dbc0787af2	
/172.21.0.2	app-1						
orders-app	orders	2	3	3	0	app-1-20cddbc5-1f06-4544-bab3-65dbc0787af2	
/172.21.0.2	app-1						

- One member owns all three partitions.
- The group is new, so it started at `latest` (the default `auto.offset.reset`) and committed that position: `LAG 0`, nothing printed.
- The `CONSUMER-ID` is the client ID plus a random suffix, given by the group coordinator.

Exercise 7.2 · Scale out to three consumers

Task: start two more consumers in the same group - `app-2` on `kafka2` and `app-3` on `kafka3`. Describe the members again. Then produce three orders for heidi, ivan and judy and see which consumer prints what.

✓ SOLUTION

Terminal B and Terminal C:

```
docker exec -it kafka2 kafka-console-consumer.sh --bootstrap-server kafka1:9092 --topic orders `
--group orders-app --command-property client.id=app-2 `
--formatter-property print.partition=true --formatter-property print.key=true
```

```
docker exec -it kafka3 kafka-console-consumer.sh --bootstrap-server kafka1:9092 --topic orders `
--group orders-app --command-property client.id=app-3 `
--formatter-property print.partition=true --formatter-property print.key=true
```

Terminal D:

```
kafka-consumer-groups.sh --bootstrap-server kafka1:9092 --describe --group orders-app --members --
verbose
```

GROUP	CONSUMER-ID	HOST	CLIENT-ID	#PARTITIONS	CURRENT-
ASSIGNMENT					
orders-app	app-2-fc658f39-b846-4075-aca8-6cfd73036b7d	/172.21.0.3	app-2	1	orders:1
orders-app	app-3-5a516243-b444-475c-9f77-e5a8f060ba58	/172.21.0.4	app-3	1	orders:2
orders-app	app-1-20cddbc5-1f06-4544-bab3-65dbc0787af2	/172.21.0.2	app-1	1	orders:0

When `app-2` and `app-3` joined, the group **rebalanced**: `app-1` gave up partitions 1 and 2, and each member now owns exactly one. Each partition is read by **exactly one** member of the group - that is how a group shares work without reading anything twice.

```
printf 'heidi:{"order":2013,"item":"ssd","amount":90}\nivan:
{"order":2014,"item":"ram","amount":70}\njudy:{"order":2015,"item":"fan","amount":25}\n' \
| kafka-console-producer.sh --bootstrap-server kafka1:9092 --topic orders \
--reader-property parse.key=true --reader-property key.separator=:
```

Terminal B (app-2) prints:

```
Partition:1 heidi {"order":2013,"item":"ssd","amount":90}
```

Terminal C (app-3) prints:

```
Partition:2 ivan {"order":2014,"item":"ram","amount":70}
Partition:2 judy {"order":2015,"item":"fan","amount":25}
```

Terminal A (app-1, partition 0) prints nothing: none of the three keys hashes to partition 0.

Exercise 7.3 · A fourth consumer

Task: start a fourth consumer, `app-4`, in the group (in another terminal, on `kafka1`). What does it get? Check the

group's state.

✓ SOLUTION

POWERSHELL

```
docker exec -it kafka1 kafka-console-consumer.sh --bootstrap-server kafka1:9092 --topic orders `
--group orders-app --command-property client.id=app-4
```

KAFKA1 SHELL

```
kafka-consumer-groups.sh --bootstrap-server kafka1:9092 --describe --group orders-app --members --
verbose
```

OUTPUT

GROUP	CONSUMER-ID	HOST	CLIENT-ID	#PARTITIONS	CURRENT-
ASSIGNMENT					
orders-app	app-2-fc658f39-b846-4075-aca8-6cfd73036b7d	/172.21.0.3	app-2	1	orders:1
orders-app	app-4-df6fa62f-f346-411a-841b-044d924c7078	/172.21.0.2	app-4	0	-
orders-app	app-3-5a516243-b444-475c-9f77-e5a8f060ba58	/172.21.0.4	app-3	1	orders:2
orders-app	app-1-20cddbc5-1f06-4544-bab3-65dbc0787af2	/172.21.0.2	app-1	1	orders:0

app-4 has **no partitions**: a partition cannot be split between members, so a group can use at most as many consumers as the topic has partitions. The extra one waits as a hot standby.

KAFKA1 SHELL

```
kafka-consumer-groups.sh --bootstrap-server kafka1:9092 --describe --group orders-app --state
```

OUTPUT

GROUP	COORDINATOR (ID)	ASSIGNMENT-STRATEGY	STATE	#MEMBERS
orders-app	kafka1:9092 (1)	range	Stable	4

Field Meaning

COORDINATOR The broker managing this group - the leader of the `__consumer_offsets` partition that stores the group (Exercise 7.8)

ASSIGNMENT-STRATEGY How partitions were split. `range` is the Java consumer's first default choice.

STATE `Stable` (working), `PreparingRebalance` / `CompletingRebalance` (rebalancing), `Empty` (no members but offsets kept), `Dead` (being removed)

Exercise 7.4 · A consumer fails

Task: stop `app-2` (Ctrl+C in terminal B). What happens to partition 1?

✓ SOLUTION

KAFKA1 SHELL

```
kafka-consumer-groups.sh --bootstrap-server kafka1:9092 --describe --group orders-app --members --
verbose
```

OUTPUT

GROUP	CONSUMER-ID	HOST	CLIENT-ID	#PARTITIONS	CURRENT-
ASSIGNMENT					
orders-app	app-4-df6fa62f-f346-411a-841b-044d924c7078	/172.21.0.2	app-4	1	orders:2
orders-app	app-3-5a516243-b444-475c-9f77-e5a8f060ba58	/172.21.0.4	app-3	1	orders:1
orders-app	app-1-20cddbc5-1f06-4544-bab3-65dbc0787af2	/172.21.0.2	app-1	1	orders:0

The group rebalanced again and the standby `app-4` was put to work. With the classic protocol and the `range` assignor, **every** member's assignment is recalculated - here `app-3` moved from partition 2 to partition 1, and `app-4` took partition 2. The new consumer continues from the last **committed** offset of each partition, so nothing is lost. Messages processed but not yet committed before the failure are read again (at-least-once).

A clean stop (Ctrl+C) leaves the group at once. A crashed consumer is only noticed after `session.timeout.ms` (45 s) without heartbeats.

Exercise 7.5 · Lag

Task: stop **all** the consumers. Produce 300 messages to `orders`, then describe the group.

✓ SOLUTION

Press Ctrl+C in every consumer terminal. Then in terminal D:

```
kafka-consumer-groups.sh --bootstrap-server kafka1:9092 --describe --group orders-app --state
```

OUTPUT

Consumer group 'orders-app' has no active members.

GROUP	COORDINATOR (ID)	ASSIGNMENT-STRATEGY	STATE	#MEMBERS
orders-app	kafka1:9092 (1)	-	Empty	0

```
kafka-producer-perf-test.sh --bootstrap-server kafka1:9092 --topic orders --num-records 300 --record-size 50 \
  --throughput -1 --command-property acks=all
kafka-consumer-groups.sh --bootstrap-server kafka1:9092 --describe --group orders-app
```

OUTPUT

300 records sent, 560.747664 records/sec (0.03 MB/sec), 29.39 ms avg latency, 492.00 ms max latency, ...

Consumer group 'orders-app' has no active members.

GROUP	TOPIC	PARTITION	CURRENT-OFFSET	LOG-END-OFFSET	LAG	CONSUMER-ID	HOST	CLIENT-ID
orders-app	orders	0	6	6	0	-	-	-
orders-app	orders	1	4	4	0	-	-	-
orders-app	orders	2	5	305	300	-	-	-

- The group is **Empty** but its committed offsets are kept (for 7 days, `offsets.retention.minutes`), so it can resume later.
- All 300 messages went to partition 2: the perf-test messages have no key, and the sticky partitioner filled one partition. Its **LAG is 300**.
- Lag is *the* health metric for consumers. Growing lag means the group cannot keep up: add consumers (up to the number of partitions) or make processing faster.

Exercise 7.6 · Reset offsets

Task: the team wants to reprocess `orders`. With the group stopped:

1. Preview a reset to the earliest offsets, then do it.
2. Move partition 0 forward by 3.

3. Skip everything - jump to the latest offsets.
4. Set partition 1 to offset 1.
5. Preview a reset to "10 minutes ago", both by duration and by date-time.
6. Try a reset while a consumer is running.

✓ SOLUTION

Every reset needs a scope (`--topic`, `--topic orders:0` for one partition, or `--all-topics`), a target, and either `--dry-run` (show only) or `--execute`.

1 - Earliest. Always dry-run first:

```
kafka-consumer-groups.sh --bootstrap-server kafka1:9092 --group orders-app --topic orders \
  --reset-offsets --to-earliest --dry-run
```

KAFKA1 SHELL

GROUP	TOPIC	PARTITION	NEW-OFFSET
orders-app	orders	0	0
orders-app	orders	1	0
orders-app	orders	2	0

OUTPUT

Replace `--dry-run` with `--execute` to apply it - the output is the same table.

2 - Shift one partition (`orders:0` means partition 0 only; a negative number moves back):

```
kafka-consumer-groups.sh --bootstrap-server kafka1:9092 --group orders-app --topic orders:0 \
  --reset-offsets --shift-by 3 --execute
```

KAFKA1 SHELL

GROUP	TOPIC	PARTITION	NEW-OFFSET
orders-app	orders	0	3

OUTPUT

3 - Latest (skip the backlog):

```
kafka-consumer-groups.sh --bootstrap-server kafka1:9092 --group orders-app --all-topics \
  --reset-offsets --to-latest --execute
```

KAFKA1 SHELL

GROUP	TOPIC	PARTITION	NEW-OFFSET
orders-app	orders	0	6
orders-app	orders	1	4
orders-app	orders	2	305

OUTPUT

4 - An exact offset:

```
kafka-consumer-groups.sh --bootstrap-server kafka1:9092 --group orders-app --topic orders:1 \
  --reset-offsets --to-offset 1 --execute
```

KAFKA1 SHELL

GROUP	TOPIC	PARTITION	NEW-OFFSET
orders-app	orders	1	1

OUTPUT

5 - By time. `--by-duration` takes an ISO-8601 duration (`PT10M` = 10 minutes, `P1D` = 1 day); `--to-datetime` takes `YYYY-MM-DDTHH:mm:ss.sss` (UTC, or add a zone like `+05:30`):

KAFKA1 SHELL

```
kafka-consumer-groups.sh --bootstrap-server kafka1:9092 --group orders-app --topic orders \
  --reset-offsets --by-duration PT10M --dry-run
kafka-consumer-groups.sh --bootstrap-server kafka1:9092 --group orders-app --topic orders \
  --reset-offsets --to-datetime 2026-09-27T12:00:00.000 --dry-run
```

OUTPUT

GROUP	TOPIC	PARTITION	NEW-OFFSET
orders-app	orders	0	5
orders-app	orders	1	3
orders-app	orders	2	3

Kafka uses each partition's **time index** (Lab 9) to find the first message at or after that moment: the 300 perf-test messages and the latest orders.

6 - While the group is running (start a consumer in terminal A first):

KAFKA1 SHELL

```
kafka-consumer-groups.sh --bootstrap-server kafka1:9092 --group orders-app --topic orders \
  --reset-offsets --to-earliest --execute
```

OUTPUT

```
Error: Assignments can only be reset if the group 'orders-app' is inactive, but the current state is Stable.
```

Offsets can only be reset while the group is **Empty** - otherwise the running consumers would overwrite them with their next commit. Stop the consumers first.

Other targets: `--to-current` (commit the current position), `--from-file` (a CSV of `topic,partition,offset`), and `--export` to save a plan as CSV.

Exercise 7.7 · Delete offsets and groups

Task: remove `orders-reporting`'s offsets for `orders`, then delete the whole `orders-app` group.

✓ SOLUTION

KAFKA1 SHELL

```
kafka-consumer-groups.sh --bootstrap-server kafka1:9092 --delete-offsets --group orders-reporting --
topic orders
```

OUTPUT

```
Request succeeded for deleting offsets from group orders-reporting.
```

TOPIC	PARTITION	STATUS
orders	0	Successful
orders	1	Successful
orders	2	Successful

Deleting a group that still has members fails:

OUTPUT

```
Error: Deletion of some consumer groups failed:
* Group 'orders-app' could not be deleted due to: org.apache.kafka.common.errors.GroupNotEmptyException: The
group is not empty.
```

With every consumer stopped:

KAFKA1 SHELL

```
kafka-consumer-groups.sh --bootstrap-server kafka1:9092 --delete --group orders-app
```

```
kafka-consumer-groups.sh --bootstrap-server kafka1:9092 --list
```

OUTPUT

```
Deletion of requested consumer groups ('orders-app') was successful.
console-consumer-15844
orders-reporting
console-consumer-83412
console-consumer-50275
```

The `console-consumer-NNNNN` groups were created by earlier console consumers run *without* `--group`: the tool invents a group name each time. They are harmless and expire after 7 days without members.

Exercise 7.8 · Where offsets live: `__consumer_offsets`

Task: find the internal topic that stores committed offsets, and read the raw commit records of group `orders-reporting`.

✓ SOLUTION

KAFKA1 SHELL

```
kafka-topics.sh --bootstrap-server kafka1:9092 --describe --topic __consumer_offsets | head -4
```

OUTPUT

```
Topic: __consumer_offsets TopicId: nmQYx_6ZThG2B5qpu-IUSw PartitionCount: 50 ReplicationFactor: 3 Configs:
compression.type=producer,min.insync.replicas=2,cleanup.policy=compact,segment.bytes=104857600
  Topic: __consumer_offsets Partition: 0 Leader: 1 Replicas: 1,2,3 Isr: 1,2,3
  Topic: __consumer_offsets Partition: 1 Leader: 2 Replicas: 2,3,1 Isr: 1,2,3
  Topic: __consumer_offsets Partition: 2 Leader: 3 Replicas: 3,1,2 Isr: 1,2,3
```

- 50 partitions, replication factor 3, **compacted** (Lab 4): only the latest commit per group + topic + partition is kept.
- It was created automatically the first time a consumer group committed.
- A group lives in partition `abs(hashCode(group.id)) % 50`. For `orders-reporting` that is partition **44**. The leader of that partition is the group's **coordinator**.

The console consumer can decode it with a special formatter. The formatter prints the records one after another **without line breaks**, so `sed` splits them before `grep` picks out our group:

KAFKA1 SHELL

```
kafka-console-consumer.sh --bootstrap-server kafka1:9092 --topic __consumer_offsets --from-beginning \
  --formatter org.apache.kafka.tools.consumer.OffsetsMessageFormatter --timeout-ms 8000 2>/dev/null \
  | sed 's/{\"key\"/}/\n{\"key\"/g' | grep '\"group\":\"orders-reporting\"'
```

OUTPUT

```
{\"key\":{\"type\":1,\"data\":{\"group\":\"orders-reporting\",\"topic\":\"orders\",\"partition\":0}},\"value\":
{\"version\":4,\"data\":
{\"offset\":5,\"leaderEpoch\":-1,\"metadata\":\"\",\"commitTimestamp\":1790510722450,\"topicId\":\"ywkGJzfISRGp0UN2q1Y88A\"}
}}
{\"key\":{\"type\":1,\"data\":{\"group\":\"orders-reporting\",\"topic\":\"orders\",\"partition\":1}},\"value\":
{\"version\":4,\"data\":
{\"offset\":0,\"leaderEpoch\":-1,\"metadata\":\"\",\"commitTimestamp\":1790510722450,\"topicId\":\"ywkGJzfISRGp0UN2q1Y88A\"}
}}
{\"key\":{\"type\":1,\"data\":{\"group\":\"orders-reporting\",\"topic\":\"orders\",\"partition\":2}},\"value\":
{\"version\":4,\"data\":
{\"offset\":0,\"leaderEpoch\":-1,\"metadata\":\"\",\"commitTimestamp\":1790510722450,\"topicId\":\"ywkGJzfISRGp0UN2q1Y88A\"}
}}
{\"key\":{\"type\":1,\"data\":{\"group\":\"orders-reporting\",\"topic\":\"orders\",\"partition\":0}},\"value\":null}
{\"key\":{\"type\":1,\"data\":{\"group\":\"orders-reporting\",\"topic\":\"orders\",\"partition\":1}},\"value\":null}
{\"key\":{\"type\":1,\"data\":{\"group\":\"orders-reporting\",\"topic\":\"orders\",\"partition\":2}},\"value\":null}
```

- The first three records are the commits from Exercise 6.5: offsets 5, 0, 0.
- The last three have `"value":null` - **tombstones** written by `--delete-offsets` in Exercise 7.7. Compaction will remove both the commits and the tombstones.
- Key `type:1` is an offset commit; `type:2` records hold group metadata (members and assignment). Lab 8 decodes this topic straight from the file.

Exercise 7.9 · The new consumer protocol (KIP-848)

Task: run two consumers in group `orders-v2` with `group.protocol=consumer`. Compare the group with a classic one.

✓ SOLUTION

In two terminals:

```
docker exec -it kafka1 kafka-console-consumer.sh --bootstrap-server kafka1:9092 --topic orders `
--group orders-v2 --command-property group.protocol=consumer --command-property client.id=v2-1
```

POWERSHELL

```
docker exec -it kafka1 kafka-console-consumer.sh --bootstrap-server kafka1:9092 --topic orders `
--group orders-v2 --command-property group.protocol=consumer --command-property client.id=v2-2
```

POWERSHELL

```
kafka-groups.sh --bootstrap-server kafka1:9092 --list
```

KAFKA1 SHELL

GROUP	TYPE	PROTOCOL
console-consumer-15844	Classic	consumer
orders-v2	Consumer	consumer
orders-reporting	Classic	consumer

OUTPUT

```
kafka-consumer-groups.sh --bootstrap-server kafka1:9092 --describe --group orders-v2 --members --verbose
kafka-consumer-groups.sh --bootstrap-server kafka1:9092 --describe --group orders-v2 --state
```

KAFKA1 SHELL

GROUP	CONSUMER-ID	HOST	CLIENT-ID	#PARTITIONS	CURRENT-EPOCH	CURRENT-ASSIGNMENT	OUTPUT
orders-v2	qiXasRWeQ_-BzAC9z77bcw	/172.21.0.2	v2-1	2	3	orders:0,1	3
orders-v2	zeFSFJpbQ0ewIausgTNLHA	/172.21.0.2	v2-2	1	3	orders:2	3

GROUP	COORDINATOR (ID)	ASSIGNMENT-STRATEGY	STATE	#MEMBERS
orders-v2	kafka3:9092 (3)	uniform	Stable	2

OUTPUT

Classic protocol

New consumer protocol (KIP-848)

Who assigns partitions

The group leader - one of the consumers

The **broker** (group coordinator)

Rebalance

Stop-the-world: every member pauses and rejoins

Incremental: only the partitions that move are paused

Epochs

-

`CURRENT-EPOCH` / `TARGET-EPOCH` show each member converging on the target assignment

	Classic protocol	New consumer protocol (KIP-848)
Default assignor	<code>range</code> (client side)	<code>uniform</code> (server side)
Enable	default	<code>group.protocol=consumer</code> in the consumer (Kafka 4.0+)

`kafka-groups.sh` is the newer tool that lists every kind of group: `Classic` and `Consumer` groups, and also `Share` and `Streams` groups.

Check yourself

1. A topic has 6 partitions. How many consumers in one group can do useful work? What do the others do?
2. Two applications must both process every order. How do you configure them?
3. Why must a group be `Empty` before you reset its offsets?

✓ ANSWERS

1. Six, one partition each. Extra members get no partitions and wait as standbys.
2. Give them **different** `group.id`s. Each group gets every message; within a group, each message goes to one member.
3. Active members hold their own positions and commit them regularly; they would overwrite the reset straight away.

Lab 8 · Inside the Log Files

Goal: open Kafka's `.log` files with `kafka-dump-log.sh` and understand exactly what is stored: record batches, records, producer IDs and sequence numbers, compression, tombstones - and the two internal logs, `__consumer_offsets` and `__cluster_metadata`.

`kafka-dump-log.sh` reads segment files **directly from disk**, so it must run on a broker that has a copy of the partition. Every broker has every `orders` partition, so we stay on `kafka1`.

Exercise 8.1 · The files of one partition

Task: list the files of `orders` partition 0 and read the two small text files.

✓ SOLUTION

```
ls -l /var/lib/kafka/data/orders-0/
```

KAFKA1 SHELL

```
-rw-r--r-- 1 appuser appuser 10485760 Sep 27 11:51 00000000000000000000.index
-rw-r--r-- 1 appuser appuser      535 Sep 27 12:02 00000000000000000000.log
-rw-r--r-- 1 appuser appuser 10485756 Sep 27 11:51 00000000000000000000.timeindex
-rw-r--r-- 1 appuser appuser      8 Sep 27 11:53 leader-epoch-checkpoint
-rw-r--r-- 1 appuser appuser    43 Sep 27 11:51 partition.metadata
```

OUTPUT

```
cat /var/lib/kafka/data/orders-0/partition.metadata
cat /var/lib/kafka/data/orders-0/leader-epoch-checkpoint
```

KAFKA1 SHELL

```
version: 0
topic_id: ywkgJzfISRGP0UN2q1Y88A
0
1
0 0
```

OUTPUT

- `partition.metadata` ties the directory to the **topic ID** (compare `kafka-topics.sh --describe`). If a topic is deleted and recreated with the same name, a leftover directory with the old ID is recognised as stale.
- `leader-epoch-checkpoint`: format version `0`, then `1` entry, then `epoch 0 started at offset 0`. Each new leader of the partition adds a line. When a failed leader comes back, it uses these epochs to find where its log diverged from the new leader's and truncates the difference.
- One segment only: 535 bytes of data - far from the 1 GB `segment.bytes` - so there is still one, active segment.

Exercise 8.2 · Record batches

Task: dump the `orders-0` segment and explain each field of a batch.

✓ SOLUTION

```
kafka-dump-log.sh --files /var/lib/kafka/data/orders-0/00000000000000000000.log
```

KAFKA1 SHELL

OUTPUT

```
Dumping /var/lib/kafka/data/orders-0/00000000000000000000.log
Log starting offset: 0
baseOffset: 0 lastOffset: 3 count: 4 baseSequence: 0 lastSequence: 3 producerId: 0 producerEpoch: 0
partitionLeaderEpoch: 0 isTransactional: false isControl: false deleteHorizonMs: OptionalLong.empty position:
0 CreateTime: 1790510023056 size: 282 magic: 2 compresscodec: none crc: 123657965 invalid: true
baseOffset: 4 lastOffset: 4 count: 1 baseSequence: 0 lastSequence: 0 producerId: 1 producerEpoch: 0
partitionLeaderEpoch: 0 isTransactional: false isControl: false deleteHorizonMs: OptionalLong.empty position:
282 CreateTime: 1790510098947 size: 111 magic: 2 compresscodec: none crc: 570975783 invalid: true
baseOffset: 5 lastOffset: 5 count: 1 baseSequence: 0 lastSequence: 0 producerId: 7 producerEpoch: 0
partitionLeaderEpoch: 0 isTransactional: false isControl: false deleteHorizonMs: OptionalLong.empty position:
393 CreateTime: 1790510578789 size: 142 magic: 2 compresscodec: none crc: 2384294184 invalid: true
```

Kafka never stores single messages: it stores **record batches**. By default `kafka-dump-log.sh` prints one line per batch.

Field	Meaning	Here
<code>baseOffset</code> / <code>lastOffset</code> / <code>count</code>	Offsets of the first and last record in the batch, and how many	Batch 1 = offsets 0-3: the four <code>orders.txt</code> lines for this partition, sent together
<code>producerId</code> / <code>producerEpoch</code>	ID the broker gave the (idempotent) producer, and its epoch	Three batches from three producer runs: IDs 0, 1 and 7
<code>baseSequence</code> / <code>lastSequence</code>	Per-producer, per-partition sequence numbers	The broker rejects a batch whose sequence it has already seen: this is how idempotence drops duplicate retries
<code>partitionLeaderEpoch</code>	The leader epoch when the batch was written	0 - these were written before any leader change
<code>isTransactional</code> / <code>isControl</code>	Part of a transaction / a transaction commit or abort marker	No transactions here
<code>deleteHorizonMs</code>	Set by compaction on batches holding tombstones	Empty (not a compacted topic)
<code>position</code>	Byte position of the batch in the <code>.log</code> file	0, 282, 393 - each batch starts where the previous one ended (0 + 282 = 282; 282 + 111 = 393)
<code>CreateTime</code>	The batch's largest timestamp	
<code>size</code>	Bytes on disk, including the 61-byte batch header	282 + 111 + 142 = 535 = the file size
<code>magic</code>	Record format version. 2 has been current since Kafka 0.11.	2
<code>compresscodec</code>	How the batch is compressed	none
<code>crc</code> / <code>invalid</code>	CRC-32C checksum of the batch, and whether it matches	Kafka checks it on write and consumers check it on read

Exercise 8.3 · The records inside

Task: print the records too, with keys, values and headers.

✓ SOLUTION

KAFKA1 SHELL

```
kafka-dump-log.sh --files /var/lib/kafka/data/orders-0/00000000000000000000.log --print-data-log
```

OUTPUT

```
baseOffset: 0 lastOffset: 3 count: 4 baseSequence: 0 lastSequence: 3 producerId: 0 ... position: 0 CreateTime:
1790510023056 size: 282 magic: 2 compresscodec: none crc: 123657965 isvalid: true
| offset: 0 CreateTime: 1790510023037 keySize: 5 valueSize: 44 sequence: 0 headerKeys: [] key: alice payload:
{"order":2001,"item":"keyboard","amount":45}
| offset: 1 CreateTime: 1790510023055 keySize: 5 valueSize: 43 sequence: 1 headerKeys: [] key: alice payload:
{"order":2004,"item":"headset","amount":60}
| offset: 2 CreateTime: 1790510023056 keySize: 3 valueSize: 40 sequence: 2 headerKeys: [] key: bob payload:
{"order":2007,"item":"cable","amount":9}
| offset: 3 CreateTime: 1790510023056 keySize: 5 valueSize: 48 sequence: 3 headerKeys: [] key: alice payload:
{"order":2010,"item":"laptop stand","amount":55}
baseOffset: 4 lastOffset: 4 count: 1 baseSequence: 0 lastSequence: 0 producerId: 1 ... position: 282
CreateTime: 1790510098947 size: 111 ...
| offset: 4 CreateTime: 1790510098947 keySize: 5 valueSize: 38 sequence: 0 headerKeys: [] key: grace payload:
{"order":2011,"item":"pen","amount":3}
baseOffset: 5 lastOffset: 5 count: 1 baseSequence: 0 lastSequence: 0 producerId: 7 ... position: 393
CreateTime: 1790510578789 size: 142 ...
| offset: 5 CreateTime: 1790510578789 keySize: 5 valueSize: 44 sequence: 0 headerKeys: [source,trace] key:
grace payload: {"order":2012,"item":"notebook","amount":12}
```

- Lines starting with `|` are **records** inside the batch above them.
- `keySize` / `valueSize` are in bytes (`alice` = 5). A null key or value shows as `-1`.
- `headerKeys: [source,trace]` - the headers from Exercise 5.3. (Header values are not printed.)
- Each record keeps its own timestamp; the batch's `CreateTime` is the largest of them.
- Inside the file, records store their offset and timestamp as **deltas** from the batch's base values. That is why batching saves space as well as network round trips.

Exercise 8.4 · Compressed batches

Task: dump the gzip-compressed topic from Exercise 5.6, first as batches, then record by record.

✓ SOLUTION

KAFKA1 SHELL

```
kafka-dump-log.sh --files /var/lib/kafka/data/comp-gzip-0/00000000000000000000.log | sed -n '1,4p'
```

OUTPUT

```
Dumping /var/lib/kafka/data/comp-gzip-0/00000000000000000000.log
Log starting offset: 0
baseOffset: 0 lastOffset: 59 count: 60 baseSequence: 0 lastSequence: 59 producerId: 11 ... position: 0
CreateTime: 1790510641963 size: 1273 magic: 2 compresscodec: gzip crc: 3797260463 isvalid: true
baseOffset: 60 lastOffset: 351 count: 292 baseSequence: 60 lastSequence: 351 producerId: 11 ... position: 1273
CreateTime: 1790510641991 size: 4523 magic: 2 compresscodec: gzip crc: 1264871835 isvalid: true
```

The second batch holds **292 records in 4,523 bytes** - about 15 bytes per record, for JSON lines of about 110 bytes. `compresscodec: gzip` confirms the codec. The broker stored the producer's compressed batch unchanged.

To see the records, the tool must decompress the batch. `--deep-iteration` does that (it is implied by `--print-data-log`):

KAFKA1 SHELL

```
kafka-dump-log.sh --files /var/lib/kafka/data/comp-gzip-0/00000000000000000000.log --deep-iteration --
print-data-log | sed -n '1,5p'
```

OUTPUT

```
Dumping /var/lib/kafka/data/comp-gzip-0/00000000000000000000.log
Log starting offset: 0
```

```
baseOffset: 0 lastOffset: 59 count: 60 ... compresscodec: gzip crc: 3797260463 invalid: true
| offset: 0 CreateTime: 1790510641932 keySize: -1 valueSize: 107 sequence: 0 headerKeys: [] payload: {"order":
3178, "customer": "cust-47", "item": "chair", "amount": 81, "city": "Delhi", "status": "CREATED"}
| offset: 1 CreateTime: 1790510641956 keySize: -1 valueSize: 111 sequence: 1 headerKeys: [] payload: {"order":
3396, "customer": "cust-33", "item": "monitor", "amount": 32, "city": "Kolkata", "status": "CREATED"}
```

`keySize: -1`: the perf-test producer sends no key.

Exercise 8.5 · A compacted segment

Task: dump the compacted `customer-profile` segment from Exercise 4.4. How are deletions and removed offsets shown?

✓ SOLUTION

```
kafka-dump-log.sh --files /var/lib/kafka/data/customer-profile-0/00000000000000000000.log --print-data-
log
```

OUTPUT

```
Dumping /var/lib/kafka/data/customer-profile-0/00000000000000000000.log
Log starting offset: 0
baseOffset: 0 lastOffset: 5 count: 3 baseSequence: 0 lastSequence: 5 producerId: 3 ... deleteHorizonMs:
OptionalLong[1790510504371] position: 0 ... size: 167 ... compresscodec: none ...
| offset: 3 CreateTime: 1790510459930 keySize: 5 valueSize: 32 sequence: 3 headerKeys: [] key: carol payload:
{"city":"Delhi","tier":"silver"}
| offset: 4 CreateTime: 1790510459930 keySize: 5 valueSize: 34 sequence: 4 headerKeys: [] key: alice payload:
{"city":"Bengaluru","tier":"gold"}
| offset: 5 CreateTime: 1790510459930 keySize: 3 valueSize: -1 sequence: 5 headerKeys: [] key: bob
Non-consecutive offsets in /var/lib/kafka/data/customer-profile-0/00000000000000000000.log
-1 is followed by 3
```

- The batch still says `baseOffset: 0 lastOffset: 5`, but `count: 3`: the cleaner kept the batch's offset range and removed three records from inside it.
- Offsets 0, 1 and 2 are simply missing. The tool warns about the gap ("-1 is followed by 3": the first record is 3). Gaps are normal in compacted topics.
- bob's record has `valueSize: -1` and no payload: a **tombstone**.
- `deleteHorizonMs` is now set: after that time (now + `delete.retention.ms`), the next cleaning may remove the tombstone as well.

Exercise 8.6 · Decode `__consumer_offsets` from disk

Task: decode the partition of `__consumer_offsets` that holds group `orders-reporting` (partition 44, see Exercise 7.8).

✓ SOLUTION

`--offsets-decoder` turns the binary keys and values into JSON:

```
kafka-dump-log.sh --files /var/lib/kafka/data/__consumer_offsets-44/00000000000000000000.log --offsets-
decoder \
| grep orders-reporting | head -6
```

OUTPUT

```
| offset: 4 ... key: {"type":"2","data":{"group":"orders-reporting"}} payload: {"version":"3","data":
{"protocolType":"","generation":0,"protocol":null,"leader":null,"currentStateTimestamp":1790510722280,"members
":[]}}
| offset: 5 ... key: {"type":"2","data":{"group":"orders-reporting"}} payload: {"version":"3","data":
{"protocolType":"consumer","generation":1,"protocol":"range","leader":"lab-consumer-1e03a0e7-...", "members":
[{"memberId":"lab-consumer-1e03a0e7-...", "clientId":"lab-
consumer","clientHost":"/172.21.0.2","rebalanceTimeout":300000,"sessionTimeout":45000,"subscription":
{"topics":["orders"],...},"assignment":{"assignedPartitions":[{"topic":"orders","partitions":
[0,1,2]}],...}}]}
| offset: 6 ... key: {"type":"1","data":{"group":"orders-reporting","topic":"orders","partition":0}} payload:
{"version":"4","data":
{"offset":5,"leaderEpoch":-1,"metadata":"","commitTimestamp":1790510722450,"topicId":"ywkgJzfISRGp0UN2q1Y88A"}
}
| offset: 7 ... key: {"type":"1","data":{"group":"orders-reporting","topic":"orders","partition":1}} payload:
{"version":"4","data":{"offset":0,...}}
| offset: 8 ... key: {"type":"1","data":{"group":"orders-reporting","topic":"orders","partition":2}} payload:
{"version":"4","data":{"offset":0,...}}
| offset: 9 ... key: {"type":"2","data":{"group":"orders-reporting"}} payload: {"version":"3","data":
{"protocolType":"consumer","generation":2,"protocol":null,"leader":null,"members":[]}}
```

This is the whole life of the console consumer from Exercise 6.5, written by the group coordinator:

1. **offset 4** - group created, generation 0, no members.
2. **offset 5** - generation 1: one member (`lab-consumer`), strategy `range`, assigned `orders` 0, 1, 2.
3. **offsets 6-8** - the offset commits: 5, 0, 0 (the `CURRENT-OFFSET` column in Exercise 6.5).
4. **offset 9** - generation 2: the consumer left; the group is empty.

`type 1` = offset commit, `type 2` = group metadata. `kafka-consumer-groups.sh` simply reads the latest of these through the coordinator.

Exercise 8.7 · Decode the cluster metadata log

Task: find the record that created the `orders` topic in the KRaft metadata log, and count the record types in it.

✓ SOLUTION

KAFKA1 SHELL

```
kafka-dump-log.sh --files /var/lib/kafka/data/__cluster_metadata-0/00000000000000000000.log --cluster-
metadata-decoder \
| grep '"name":"orders"'
```

OUTPUT

```
| offset: 111 CreateTime: 1790509878206 keySize: -1 valueSize: 27 sequence: -1 headerKeys: [] payload:
{"type":"TOPIC_RECORD","version":0,"data":{"name":"orders","topicId":"ywkgJzfISRGp0UN2q1Y88A"}}
```

KAFKA1 SHELL

```
kafka-dump-log.sh --files /var/lib/kafka/data/__cluster_metadata-0/00000000000000000000.log --cluster-
metadata-decoder \
| grep -o '"type":"[A-Z_]+"' | sort | uniq -c | sort -rn
```

OUTPUT

```
2464 "type":"NO_OP_RECORD"
167 "type":"PARTITION_CHANGE_RECORD"
90 "type":"PARTITION_RECORD"
26 "type":"CONFIG_RECORD"
20 "type":"TOPIC_RECORD"
10 "type":"BROKER_REGISTRATION_CHANGE_RECORD"
6 "type":"FEATURE_LEVEL_RECORD"
4 "type":"REMOVE_TOPIC_RECORD"
4 "type":"REGISTER_CONTROLLER_RECORD"
4 "type":"REGISTER_BROKER_RECORD"
```

```
1 "type":"PRODUCER_IDS_RECORD"
1 "type":"END_TRANSACTION_RECORD"
1 "type":"CLEAR_ELR_RECORD"
1 "type":"BEGIN_TRANSACTION_RECORD"
```

Everything you did in Labs 2-7 is here as records - the same *log* idea Kafka uses for your data:

Record	Written when ...
<code>TOPIC_RECORD</code> + <code>PARTITION_RECORD</code>	A topic is created: one topic record, one partition record per partition (20 topics, 90 partitions so far)
<code>PARTITION_CHANGE_RECORD</code>	A leader or the ISR changes - Lab 3's broker failure produced many
<code>CONFIG_RECORD</code>	A config is set with <code>--config</code> or <code>kafka-configs.sh</code>
<code>REMOVE_TOPIC_RECORD</code>	A topic is deleted
<code>REGISTER_BROKER_RECORD</code> / <code>BROKER_REGISTRATION_CHANGE_RECORD</code>	A broker starts, or is fenced / unfenced
<code>NO_OP_RECORD</code>	The active controller writes one regularly so that followers can tell it is alive

BROWSE THE METADATA LIKE A FILE SYSTEM

`kafka-metadata-shell.sh` loads the metadata log into a tree you can `ls` and `cat`. It refuses to open a live broker's directory (it is locked), so work on a copy:

```
mkdir -p /tmp/meta
cp /var/lib/kafka/data/__cluster_metadata-0/00000000000000000000.log /tmp/meta/
kafka-metadata-shell.sh --snapshot /tmp/meta/00000000000000000000.log ls /image/topics/byName
kafka-metadata-shell.sh --snapshot /tmp/meta/00000000000000000000.log cat /image/topics/byName/orders/0
```

```
__consumer_offsets
audit-log
...
orders
payments
...
PartitionRegistration(replicas=[3, 1, 2], directories=[HRx9RyH0pi-j8zhurmQcgA, W6ZXRWs45VFCVynSoUEXg, 9-
duCaGAc4Vx1M-rPcTcMA], isr=[1, 2, 3], removingReplicas=[], addingReplicas=[], elr=[], lastKnownElr=[],
leader=3, leaderRecoveryState=RECOVERED, leaderEpoch=0, partitionEpoch=2)
```

Without a command at the end it opens an interactive shell (`ls`, `cd`, `cat`, `find`, `exit`).

Other decoders

Option	Decodes
<code>--offsets-decoder</code>	<code>__consumer_offsets</code>
<code>--transaction-log-decoder</code>	<code>__transaction_state</code> (transactional producers)
<code>--cluster-metadata-decoder</code>	<code>__cluster_metadata</code>
<code>--share-group-state-decoder</code>	<code>__share_group_state</code> (share groups)

Option	Decodes
<code>--remote-log-metadata-decoder</code>	<code>__remote_log_metadata</code> (tiered storage)
<code>--key-decoder-class</code> / <code>--value-decoder-class</code>	Your own topics, with a custom decoder class

Check yourself

1. A batch shows `baseOffset: 200 lastOffset: 249 count: 50 position: 81920 size: 3100`. Where does the next batch start in the file, and what is its base offset?
2. The same `producerId` and `baseSequence` arrive twice. What does the broker do, and why?
3. How can you prove, from disk alone, that a topic's data is compressed?

✓ ANSWERS

1. At byte position 85020 ($81920 + 3100$), with base offset 250 - unless the topic is compacted, where gaps are possible.
2. It acknowledges the second one without writing it again: it is a retry of a batch it already has. This is producer idempotence.
3. `kafka-dump-log.sh --files <segment>.log` shows `compresscodec:` on every batch, and `count` versus `size` shows how many records share how few bytes.

Lab 9 · Index Files

Goal: understand how Kafka finds offset N - or the first message after time T - in a segment of a gigabyte without reading it from the start: the **offset index** (`.index`) and the **time index** (`.timeindex`).

Background: two small lookup tables per segment

File	Entry	Entry size	Answers
<code>.index</code>	(relative offset, byte position in <code>.log</code>)	8 bytes: 4 + 4	"Where in the file is offset N?"
<code>.timeindex</code>	(timestamp, relative offset)	12 bytes: 8 + 4	"Which offset is the first at or after time T?"

Three design choices make them tiny and fast:

- **Sparse.** An entry is added only after about `index.interval.bytes` (4096) bytes of log have been written since the previous entry - not for every message. Kafka finds the nearest entry below the target, then scans a few KB of the log.
- **Relative offsets.** Offsets are stored as *offset minus the segment's base offset*, so they fit in 4 bytes.
- **Memory-mapped.** Index files are mapped into memory and searched with a binary search. Each is pre-allocated at `segment.index.bytes` (10 MB) while the segment is active and trimmed to its real size when the segment is rolled.

Exercise 9.1 · Build two indexed segments

Task: create `index-demo` (defaults) and `index-dense` (`index.interval.bytes=512`), each with one partition on broker 1 and 1 MB segments. Write 12,000 messages of 100 bytes to each, in small batches, and compare the index files.

✓ SOLUTION

```
kafka-topics.sh --bootstrap-server kafka1:9092 --create --topic index-demo --replica-assignment 1 \
--config segment.bytes=1048576
kafka-topics.sh --bootstrap-server kafka1:9092 --create --topic index-dense --replica-assignment 1 \
--config segment.bytes=1048576 --config index.interval.bytes=512
for t in index-demo index-dense; do
  kafka-producer-perf-test.sh --bootstrap-server kafka1:9092 --topic $t --num-records 12000 --record-
size 100 \
  --throughput 4000 --command-property acks=1 linger.ms=0 batch.size=1024 | tail -1
done
```

OUTPUT

```
12000 records sent, 3985.386915 records/sec (0.38 MB/sec), 131.40 ms avg latency, ...
12000 records sent, 3978.779841 records/sec (0.38 MB/sec), 76.54 ms avg latency, ...
```

`batch.size=1024` and `linger.ms=0` force small batches (about 8 records each), so the index gets many entries to show.

```
ls -l /var/lib/kafka/data/index-demo-0/ /var/lib/kafka/data/index-dense-0/
```

OUTPUT

```
/var/lib/kafka/data/index-demo-0/:
-rw-r--r-- 1 appuser appuser 1832 Sep 27 12:12 00000000000000000000.index
-rw-r--r-- 1 appuser appuser 1047947 Sep 27 12:12 00000000000000000000.log
-rw-r--r-- 1 appuser appuser 2520 Sep 27 12:12 00000000000000000000.timeindex
```

```
-rw-r--r-- 1 appuser appuser 10485760 Sep 27 12:12 000000000000000008932.index
-rw-r--r-- 1 appuser appuser 363997 Sep 27 12:12 000000000000000008932.log
-rw-r--r-- 1 appuser appuser 10 Sep 27 12:12 000000000000000008932.snapshot
-rw-r--r-- 1 appuser appuser 10485756 Sep 27 12:12 000000000000000008932.timeindex
...
/var/lib/kafka/data/index-dense-0/:
-rw-r--r-- 1 appuser appuser 8480 Sep 27 12:12 00000000000000000000000000000000.index
-rw-r--r-- 1 appuser appuser 1048067 Sep 27 12:12 00000000000000000000000000000000.log
-rw-r--r-- 1 appuser appuser 7824 Sep 27 12:12 00000000000000000000000000000000.timeindex
-rw-r--r-- 1 appuser appuser 10485760 Sep 27 12:12 00000000000000000000000000000000.index
...
```

- The first segment of each topic is closed (rolled at ~1 MB), so its index was **trimmed**; the active segment still has the 10 MB pre-allocated files.
- `index-demo`: **1,832 bytes / 8 = 229 entries** for a 1 MB log - one entry per ~4.6 KB.
- `index-dense`: **8,480 bytes / 8 = 1,060 entries** - about 4.6 times as many (not 8 times: see Exercise 9.2).
- Time index: 2,520 / 12 = 210 entries versus 7,824 / 12 = 652.

Exercise 9.2 · Read the offset index

Task: dump the first entries of `index-demo`'s closed `.index` file, count its entries, and look at the raw bytes.

✓ SOLUTION

```
kafka-dump-log.sh --files /var/lib/kafka/data/index-demo-0/00000000000000000000000000000000.index | head -8
```

OUTPUT

```
Dumping /var/lib/kafka/data/index-demo-0/00000000000000000000000000000000.index
offset: 47 position: 4665
offset: 87 position: 9330
offset: 127 position: 13995
offset: 167 position: 18660
offset: 207 position: 23325
offset: 247 position: 27990
offset: 287 position: 32655
```

```
for t in index-demo index-dense; do
  printf "%-12s entries in first .index: " $t
  kafka-dump-log.sh --files /var/lib/kafka/data/$t-0/00000000000000000000000000000000.index | grep -c "^offset"
done
```

OUTPUT

```
index-demo  entries in first .index: 229
index-dense  entries in first .index: 1060
```

Entries appear every 4,665 bytes and 40 offsets. Each batch here holds 8 records and takes 933 bytes (61-byte header + 8 records of about 109 bytes). After 5 batches (4,665 bytes) more than 4,096 bytes have been written since the last entry, so the 6th batch gets one. In `index-dense` every 933-byte batch crosses the 512-byte interval, so every batch after the first gets an entry (`offset: 15 position: 933`, `offset: 23 position: 1866`, ...).

The raw bytes confirm the format - 4-byte relative offset, 4-byte position, big-endian:

```
xxd -l 48 /var/lib/kafka/data/index-demo-0/00000000000000000000000000000000.index
```

OUTPUT

```
00000000: 0000 002f 0000 1239 0000 0057 0000 2472 .../...9...W...$r
00000010: 0000 007f 0000 36ab 0000 00a7 0000 48e4 .....6.....H.
00000020: 0000 00cf 0000 5b1d 0000 00f7 0000 6d56 .....[.....mV
```

`0000002f` = 47 and `00001239` = 4665: the first entry. `00000057` = 87 and `00002472` = 9330: the second. The tool adds the segment's base offset (0 here) to the relative offset when printing.

Exercise 9.3 · What does an entry point to?

Task: find the batches that start at positions 4665, 9330 and 13995 in the `.log`. What exactly does an index entry record?

✓ SOLUTION

KAFKA1 SHELL

```
kafka-dump-log.sh --files /var/lib/kafka/data/index-demo-0/00000000000000000000.log \
| grep -E 'position: (4665|9330|13995) '
```

OUTPUT

```
baseOffset: 40 lastOffset: 47 count: 8 ... position: 4665
baseOffset: 80 lastOffset: 87 count: 8 ... position: 9330
baseOffset: 120 lastOffset: 127 count: 8 ... position: 13995
```

An index entry maps **the last offset of a batch** to **the byte position where that batch starts**. Entry `offset: 47 position: 4665` means "the batch containing offset 47 (offsets 40-47) begins at byte 4665". Kafka always reads whole batches, so pointing at batch starts is all it needs.

Exercise 9.4 · Follow a lookup by hand

Task: a consumer asks for offset 100 of `index-demo`. Work out, step by step, how the broker finds it. Then check your answer with a consumer.

✓ SOLUTION

Step 1 - Pick the segment. The segments' base offsets are in their file names:

KAFKA1 SHELL

```
ls /var/lib/kafka/data/index-demo-0/*.log
```

OUTPUT

```
/var/lib/kafka/data/index-demo-0/00000000000000000000.log
/var/lib/kafka/data/index-demo-0/0000000000000000008932.log
```

The segment with the largest base offset not above 100 is `00000000000000000000`. (The broker keeps all its segments in a sorted map by base offset, so this is a quick lookup.)

Step 2 - Search that segment's `.index` for the largest entry at or below 100. From Exercise 9.2: `offset: 87 position: 9330` (the next entry, 127, is too big). This is a binary search over the memory-mapped file.

Step 3 - Scan the `.log` from byte 9330. Read batch headers forward - `80-87`, `88-95`, `96-103` - until a batch contains offset 100, and send from there. At most `index.interval.bytes` (~4 KB) is scanned.

Check:

KAFKA1 SHELL

```
kafka-console-consumer.sh --bootstrap-server kafka1:9092 --topic index-demo --partition 0 --offset 100 \
--max-messages 1 --formatter-property print.offset=true | cut -c1-60
```

OUTPUT

```
Offset:100 JZMBWWTXBSBCBKFECDXOTAYJOWYFJGIIDWWRILUZAQZPQJ
Processed a total of 1 messages
```

Kafka actually returns the *whole batch* 96-103, and the consumer drops the records before offset 100. The broker never parses individual records to serve a fetch - it copies byte ranges from the file straight to the network (zero-copy `sendfile`), which is a big part of why Kafka is fast.

Exercise 9.5 · The time index

Task: dump the first entries of the `.timeindex`, then ask Kafka for the first offset at or after timestamp `1790511121235`.

✓ SOLUTION

KAFKA1 SHELL

```
kafka-dump-log.sh --files /var/lib/kafka/data/index-demo-0/00000000000000000000.timeindex | head -8
```

OUTPUT

```
Dumping /var/lib/kafka/data/index-demo-0/00000000000000000000.timeindex
timestamp: 1790511121224 offset: 47
timestamp: 1790511121229 offset: 87
timestamp: 1790511121232 offset: 127
timestamp: 1790511121235 offset: 159
timestamp: 1790511121238 offset: 191
timestamp: 1790511121240 offset: 231
timestamp: 1790511121244 offset: 287
```

Each entry records the **largest timestamp seen so far** in the segment and the offset of the record that had it. Entries are only added when the timestamp increases, so the file is always sorted by time - even if producers send records with slightly out-of-order timestamps.

KAFKA1 SHELL

```
kafka-get-offsets.sh --bootstrap-server kafka1:9092 --topic index-demo --time 1790511121235
```

OUTPUT

```
index-demo:0:159
```

The broker (1) picks the first segment whose largest timestamp is at or after T, (2) finds the time-index entry just below T, (3) uses the offset index to jump into the `.log`, and (4) scans for the first record with timestamp at or after T - offset 159.

The same lookup is behind `kafka-consumer-groups.sh --reset-offsets --to-datetime / --by-duration` (Exercise 7.6), the Java consumer's `offsetsForTimes()`, and time-based retention, which uses each segment's largest timestamp.

Exercise 9.6 · Check an index

Task: verify that the index of `index-demo`'s first segment is consistent.

✓ SOLUTION

KAFKA1 SHELL

```
kafka-dump-log.sh --files /var/lib/kafka/data/index-demo-0/00000000000000000000.index --index-sanity-check
kafka-dump-log.sh --files /var/lib/kafka/data/index-demo-0/00000000000000000000.index --verify-index-only
```

OUTPUT

```
Dumping /var/lib/kafka/data/index-demo-0/00000000000000000000.index
/var/lib/kafka/data/index-demo-0/00000000000000000000.index passed sanity check.
Dumping /var/lib/kafka/data/index-demo-0/00000000000000000000.index
```

- `--index-sanity-check` checks the file's structure: its size is a multiple of the entry size, and the entries are in order.
- `--verify-index-only` compares every entry against the `.log`; does each position really start a batch with that offset? No output means no mismatches.

INDEXES ARE DISPOSABLE

Indexes can always be rebuilt from the `.log`, which is the only source of truth. After an unclean shutdown a broker checks the indexes of recent segments and rebuilds any that are corrupt; you may see *"Found a corrupted index file ... rebuilding index files"* in its log. Deleting a closed segment's `.index` and `.timeindex` while the broker is stopped is harmless - they are rebuilt on start (slower startup, nothing lost).

Tuning the indexes

Setting	Default	Effect
<code>index.interval.bytes</code>	4096	Smaller = more entries, bigger indexes, shorter scans. Rarely worth changing.
<code>segment.index.bytes</code>	10485760	Maximum index size. A segment also rolls when its index is full: 10 MB / 8 bytes = 1,310,720 entries.
<code>log.index.size.max.bytes</code>	10485760	Broker-level default for the above

Check yourself

1. A segment's `.index` is 4,000 bytes after rolling. How many entries does it have?
2. Why is the index sparse instead of having one entry per message?
3. A consumer asks for an offset that is below the log start offset. Does the index help?

✓ ANSWERS

1. 500 (4,000 / 8).
2. To stay small enough to keep in memory for every segment of every partition. A scan of a few KB after the lookup is cheap, and fetches return whole batches anyway.
3. No. The broker checks the offset against the log start offset first and returns `OFFSET_OUT_OF_RANGE`; the consumer then applies `auto.offset.reset`.

Lab 10 · Cheat Sheet and Troubleshooting

Every command from the labs, grouped by task. `B=kafka1:9092` stands for `--bootstrap-server kafka1:9092`.

Cluster (Windows / PowerShell, in the lab folder)

Task	Command
Start / status	<code>docker compose up -d</code> · <code>docker compose ps</code>
Shell in a broker	<code>docker exec -it kafka1 bash</code>
Broker log	<code>docker compose logs -f kafka1</code>
Stop / start one broker	<code>docker stop kafka2</code> · <code>docker start kafka2</code>
Stop, keep data	<code>docker compose down</code>
Stop, delete everything	<code>docker compose down -v</code>
With Kafka UI	<code>docker compose --profile ui up -d</code> -> <code>http://localhost:8080</code>

Cluster information (inside a broker)

Task	Command
Version	<code>kafka-topics.sh --version</code>
Cluster ID	<code>kafka-cluster.sh cluster-id --bootstrap-server B</code>
KRaft leader and voters	<code>kafka-metadata-quorum.sh --bootstrap-server B describe --status</code>
Controller replication	<code>kafka-metadata-quorum.sh --bootstrap-server B describe --replication</code>
Brokers	<code>kafka-broker-api-versions.sh --bootstrap-server B \ grep -E '^kafka'</code>
Feature versions	<code>kafka-features.sh --bootstrap-server B describe</code>

Topics

Task	Command
Create	<code>kafka-topics.sh --bootstrap-server B --create --topic T --partitions 3 --replication-factor 3</code> <code>[--config k=v]</code>
Create if missing	<code>... --create --topic T --if-not-exists</code>
Create with placement	<code>... --create --topic T --replica-assignment 1:2,2:3,3:1</code>
List	<code>kafka-topics.sh --bootstrap-server B --list [--exclude-internal]</code>
Describe	<code>kafka-topics.sh --bootstrap-server B --describe [--topic T]</code>
Health filters	<code>--describe --under-replicated-partitions</code> · <code>--under-min-isr-partitions</code> · <code>--at-min-isr-partitions</code> · <code>--unavailable-partitions</code> · <code>--topics-with-overrides</code>
Add partitions	<code>kafka-topics.sh --bootstrap-server B --alter --topic T --partitions 6</code>

Task	Command
Delete	<code>kafka-topics.sh --bootstrap-server B --delete --topic T [--if-exists]</code> (<code>--topic</code> takes a regex)
Offsets per partition	<code>kafka-get-offsets.sh --bootstrap-server B --topic T [--time -2 \ -1 \ -3 \ epoch-ms]</code>
Size on disk	<code>kafka-log-dirs.sh --bootstrap-server B --describe --topic-list T [--broker-list 1]</code>
Delete records before offset	<code>kafka-delete-records.sh --bootstrap-server B --offset-json-file F.json</code>

Configuration

Task	Command
Topic overrides	<code>kafka-configs.sh --bootstrap-server B --describe --entity-type topics --entity-name T</code>
All topic settings	<code>... --describe --entity-type topics --entity-name T --all</code>
Set	<code>kafka-configs.sh --bootstrap-server B --alter --entity-type topics --entity-name T --add-config k=v,k2=v2</code>
Remove	<code>... --alter --entity-type topics --entity-name T --delete-config k</code>
Broker settings	<code>... --describe --entity-type brokers --entity-name 1 --all</code>
Cluster-wide defaults	<code>... --describe --entity-type brokers --entity-default</code>

Replication and leadership

Task	Command
Preferred leader election	<code>kafka-leader-election.sh --bootstrap-server B --election-type preferred --all-topic-partitions</code>
Propose a move	<code>kafka-reassign-partitions.sh --bootstrap-server B --topics-to-move-json-file F --broker-list 1,3 --generate</code>
Execute / verify a move	<code>kafka-reassign-partitions.sh --bootstrap-server B --reassignment-json-file F --execute</code> then <code>--verify</code>

Producing

Task	Command
Plain lines	<code>kafka-console-producer.sh --bootstrap-server B --topic T</code>
With keys	<code>... --reader-property parse.key=true --reader-property key.separator=:</code>
With headers	<code>... --reader-property parse.key=true --reader-property parse.headers=true</code> (line: <code>h1:v1,h2:v2<TAB>key<TAB>value</code>)
Tombstones	<code>... --reader-property null.marker=NULL</code>
From a file	<code>kafka-console-producer.sh ... < /lab/data/orders.txt</code>
Producer settings	<code>--command-property acks=all</code> · <code>--command-config /lab/config/producer.properties</code>

Task	Command
Compression (console)	<code>--compression-codec lz4</code>
Load test	<code>kafka-producer-perf-test.sh --bootstrap-server B --topic T --num-records N --record-size 500 --throughput -1 --command-property acks=all</code>

Consuming

Task	Command
New messages only	<code>kafka-console-consumer.sh --bootstrap-server B --topic T</code>
From the start	<code>... --from-beginning</code>
One partition from an offset	<code>... --partition 1 --offset 42</code> (or <code>earliest</code> / <code>latest</code>)
Stop conditions	<code>--max-messages N</code> · <code>--timeout-ms T</code>
Show metadata	<code>--formatter-property print.key=true</code> (also <code>print.partition</code> , <code>print.offset</code> , <code>print.timestamp</code> , <code>print.headers</code>)
Several topics	<code>--include 'orders\ payments'</code>
In a group	<code>--group G</code>
Consumer settings	<code>--command-property k=v</code> · <code>--command-config /lab/config/consumer.properties</code>

Consumer groups

Task	Command
List	<code>kafka-consumer-groups.sh --bootstrap-server B --list</code> · <code>kafka-groups.sh --bootstrap-server B --list</code>
Offsets and lag	<code>kafka-consumer-groups.sh --bootstrap-server B --describe --group G</code>
Members	<code>... --describe --group G --members --verbose</code>
State	<code>... --describe --group G --state</code>
Reset (group stopped)	<code>... --group G --topic T --reset-offsets --to-earliest \ --to-latest \ --to-offset N \ --shift-by N \ --to-datetime 2026-09-27T12:00:00.000 \ --by-duration PT10M --dry-run \ --execute</code>
Delete offsets	<code>... --delete-offsets --group G --topic T</code>
Delete group	<code>... --delete --group G</code>
New protocol	consumer setting <code>group.protocol=consumer</code>

Files on disk

Task	Command
Batches	<code>kafka-dump-log.sh --files /var/lib/kafka/data/T-0/00000000000000000000.log</code>
Records	<code>... --print-data-log</code>

Task	Command
Offset index	<code>kafka-dump-log.sh --files ../00000000000000000000.index</code>
Time index	<code>kafka-dump-log.sh --files ../00000000000000000000.timeindex</code>
Check an index	<code>... --index-sanity-check</code> · <code>--verify-index-only</code>
Consumer offsets topic	<code>... --files ../__consumer_offsets-44/...log --offsets-decoder</code>
Metadata log	<code>... --files ../__cluster_metadata-0/...log --cluster-metadata-decoder</code>
Metadata as a tree	<code>kafka-metadata-shell.sh --snapshot /tmp/meta/00000000000000000000.log ls /image/topics/byName</code> (on a copy)

Troubleshooting

Symptom	Likely cause	Fix
<code>docker compose up</code> says <i>port is already allocated</i>	Another Kafka (or program) uses 19092/29092/39092	<code>docker ps</code> to find it; stop it, or change the host port on the left of <code>"19092:19092"</code>
Container stays <code>(health: starting)</code> , then <code>unhealthy</code>	Broker failed to start	<code>docker compose logs kafka1</code> - look for the first <code>ERROR</code>
<code>Invalid cluster.id</code> in the log	Old volumes from a cluster with another <code>CLUSTER_ID</code>	<code>docker compose down -v</code> , then <code>up -d</code>
Windows client: <i>connection refused to kafka1:9092</i>	The client used the internal listener address	Use <code>localhost:19092,localhost:29092,localhost:39092</code> from Windows
<code>UNKNOWN_TOPIC_OR_PARTITION</code> / <i>topic not present in metadata</i> when producing	The topic does not exist, and auto-creation is off in this lab	Create it with <code>kafka-topics.sh --create</code>
<code>NotEnoughReplicasException</code>	Fewer in-sync replicas than <code>min.insync.replicas</code> , with <code>acks=all</code>	<code>--describe --under-min-isr-partitions</code> ; bring the broker back
Console consumer prints nothing	It starts at <code>latest</code>	Add <code>--from-beginning</code> , or produce while it runs
Group reset fails: <i>group ... is inactive, but the current state is Stable</i>	Consumers are still running	Stop every member of the group first
<code>kafka-metadata-shell.sh</code> : <i>Unable to lock /var/lib/kafka/data</i>	It is pointed at a live broker's directory	Copy the metadata log to <code>/tmp/meta</code> and open the copy
Git Bash: paths like <code>/lab/...</code> become <code>C:/Program Files/Git/lab/...</code>	Git Bash path conversion	Use PowerShell, or <code>export MSYS_NO_PATHCONV=1</code>
<code>--property</code> , <code>--producer-property</code> or <code>--consumer-property</code> warn <code>DEPRECATED</code>	Old option names	Use <code>--reader-property</code> / <code>--formatter-property</code> / <code>--command-property</code>

Clean up

When you are done with the labs:

POWERSHELL

```
docker compose down -v
```

This removes the three containers, their network and all data volumes. The image stays cached, so the next `docker compose up -d` starts in seconds with an empty cluster.